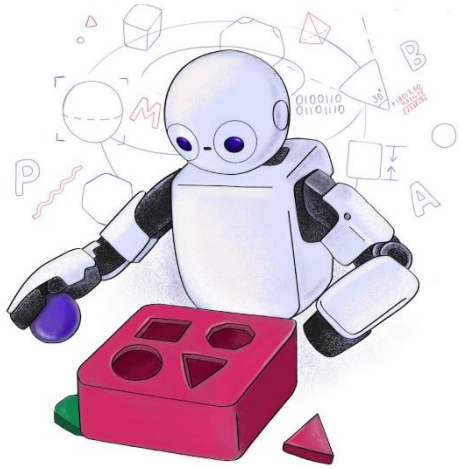
C24 – Inteligência Artificial:
Redes neurais recorrentes



Inatel

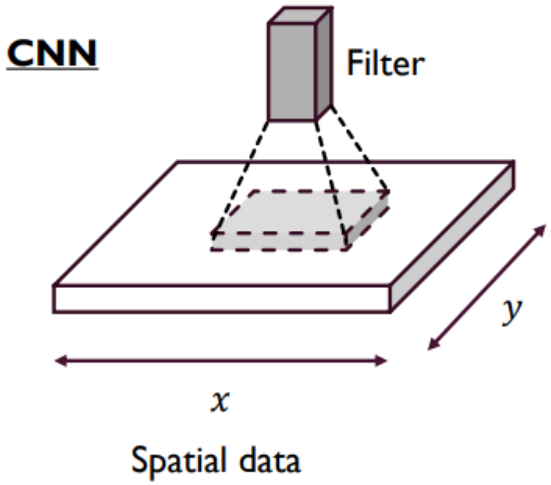
Felipe A. P. de Figueiredo
felipe.figueiredo@Inatel.br

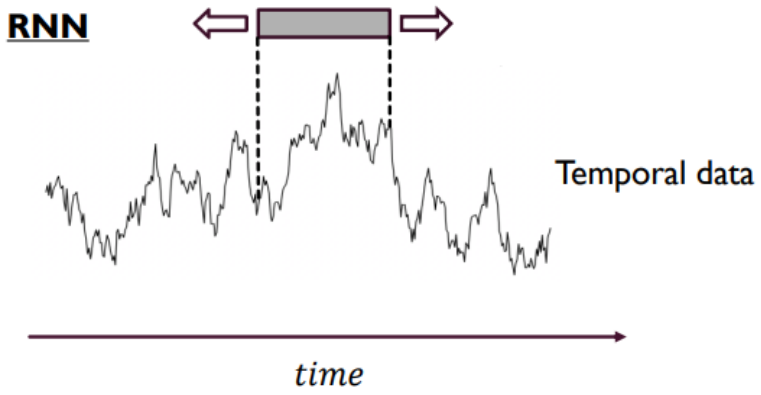
Motivação

- Muitos problemas envolvem **dados sequenciais**:
  - Voz
  - Texto
  - Séries temporais (e.g., EGC, EEG, preço de ações, desvanecimento de um canal)
  - Vídeo
- Para lidarmos com eles, precisamos **modelar dependências temporais!**
- Em outras palavras, existe **correlação temporal entre as amostras.**

Cada amostra tende a carregar informação correlacionada com as anteriores.

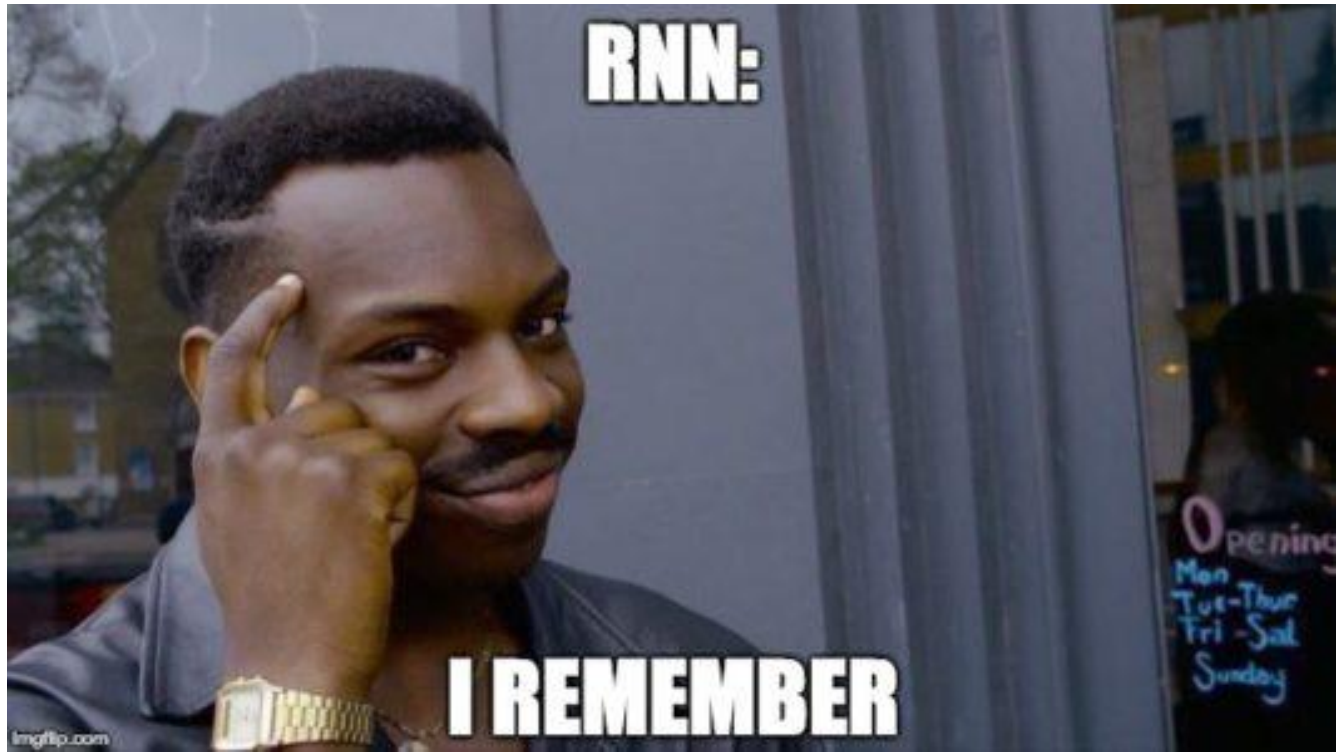
Limitação de redes tradicionais





- MLPs:
  - **Não armazenam informações de estados anteriores** (i.e., sem memória)
  - As **entradas** são tratadas de forma **independente**.
    - Isso faz com que elas **não capturem dependências temporais entre amostras**.
  - Por ser de alimentação direta, a saída **depende apenas da entrada atual**.
- CNNs:
  - Compartilham as limitações das MLPs, mas com uma exceção: elas **modelam relações espaciais**.
- Ambas famílias **exigem** que o **vetor de entrada** tenha um **tamanho predefinido e constante**.

Como podemos considerar as correlações temporais e trabalhar com entradas sem tamanho predefinido?

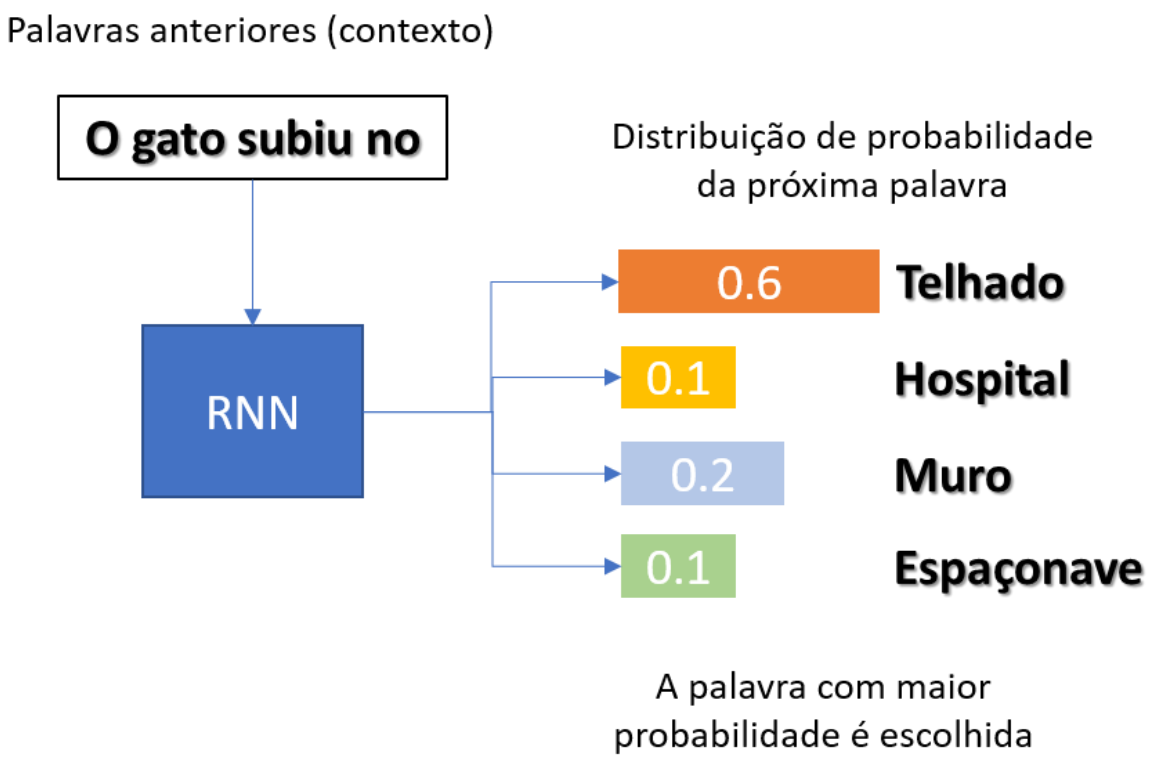


Redes neurais recorrentes (*recurrent neural networks* – RNNs) têm memória e entrada variável!

Aplicações de redes neurais recorrentes

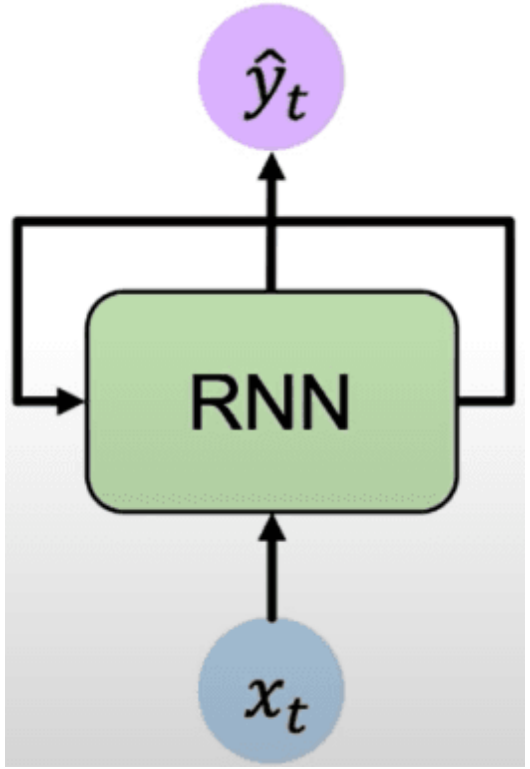
- Previsão de séries temporais (e.g., clima, ações, vendas, etc.)
- Tradução de texto
- Reconhecimento de fala
- Tradução fala para texto e vice versa
- Classificação de sentimentos
- Geração de texto
- Geração de descrições de texto para imagens (*image captioning*)
- Rastreamento de objetos
- Dentre outras

Padrões temporais



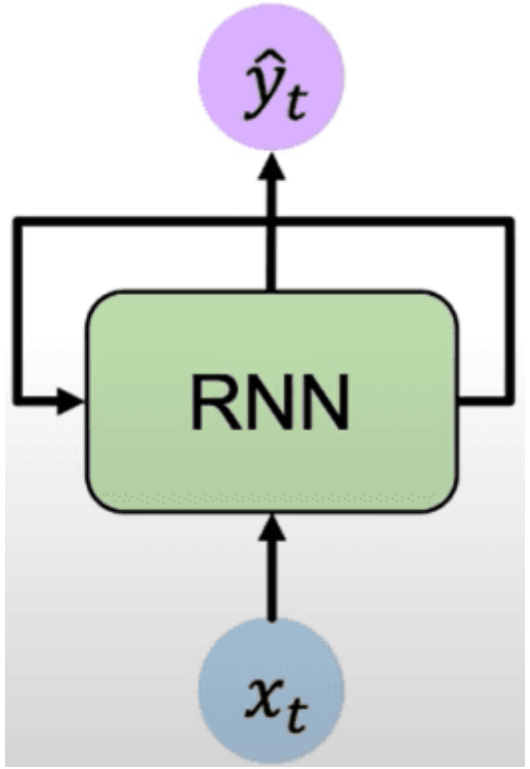
- CNNs aprendem padrões espaciais, enquanto RNNs aprendem **padrões temporais em sequências.**
- No exemplo ao lado, a RNN recebe um **contexto**: “**O gato subiu no**” e apresenta as palavras candidatas com suas respectivas probabilidades estimadas.
- A RNN **acumula** (através da memória) **informação do contexto** que faz com que a pontuação de “Telhado” seja maior.

Redes recorrentes



- Possuem **laços de realimentação**.
- Mantém um **estado interno** (**memória**) que **resume as entradas passadas**.
- O estado é atualizado a cada passo de tempo a partir da **entrada atual** e do **estado anterior**.
- A **saída** depende da **entrada atual** e do **estado interno**.

Redes recorrentes

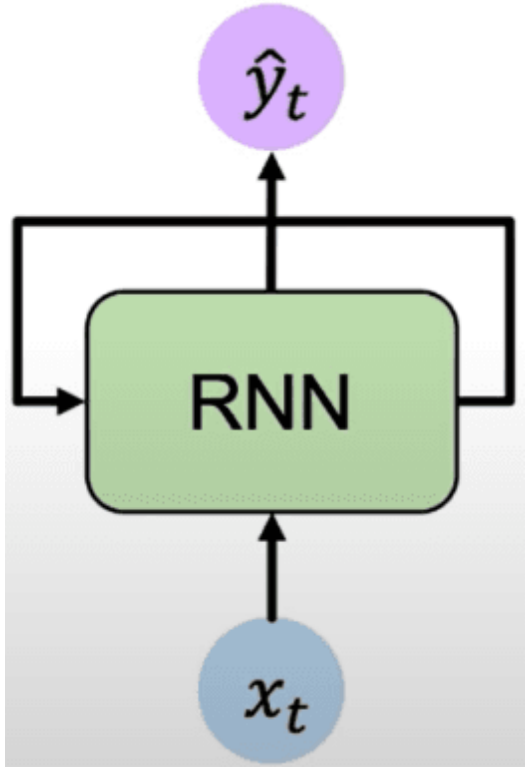


- Uma RNN lê a sequência **um elemento por vez** e **lembra do que já viu**.
- **Exemplo:** prever a próxima palavra de uma frase:

  "O gato subiu no ____"

- A rede usa as **palavras anteriores**, chamado de contexto, (*gato, subiu, no*) para atribuir probabilidades:
  - **Alta:** telhado, muro, sofá
  - **Baixa:** banana, ontem, azul

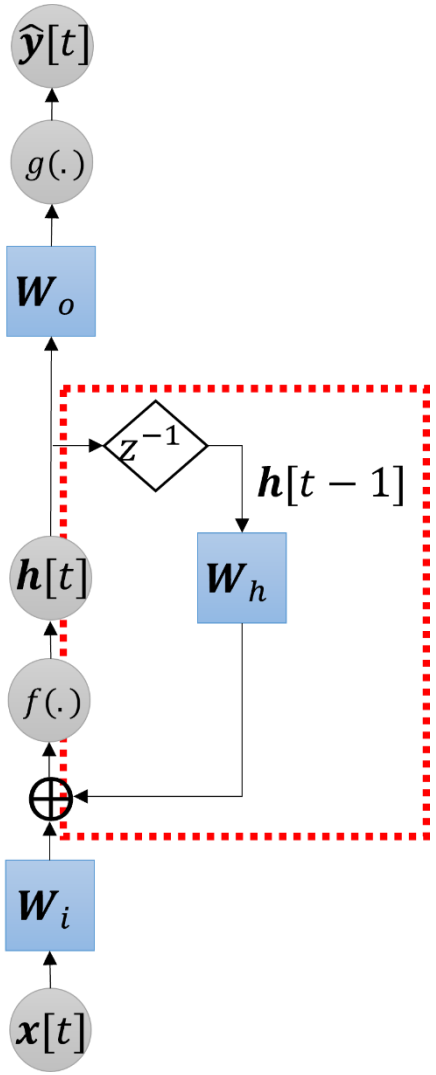
Redes recorrentes



- Uma RNN **processa sequências de qualquer comprimento**, um elemento por vez.
- A cada passo, ela **usa o que se lembra do passado** para gerar a saída.
- **Limitação:** nas RNNs simples, a informação de entradas distantes **se perde** ao longo do tempo.
  - **A memória desaparece** ao longo do tempo.
- **Exemplo**: "*O gato que foi atropelado e adotamos no ano passado subiu no ____*"

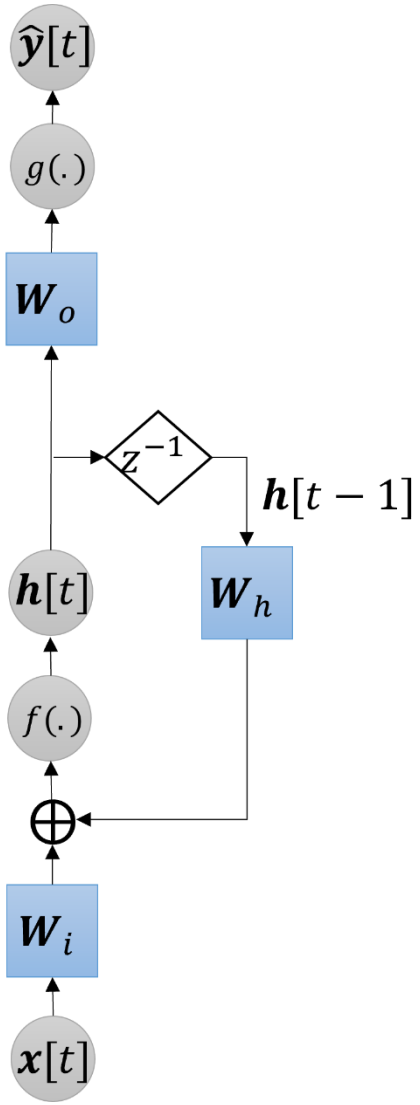
O que compõem as RNNs?

Célula recorrente



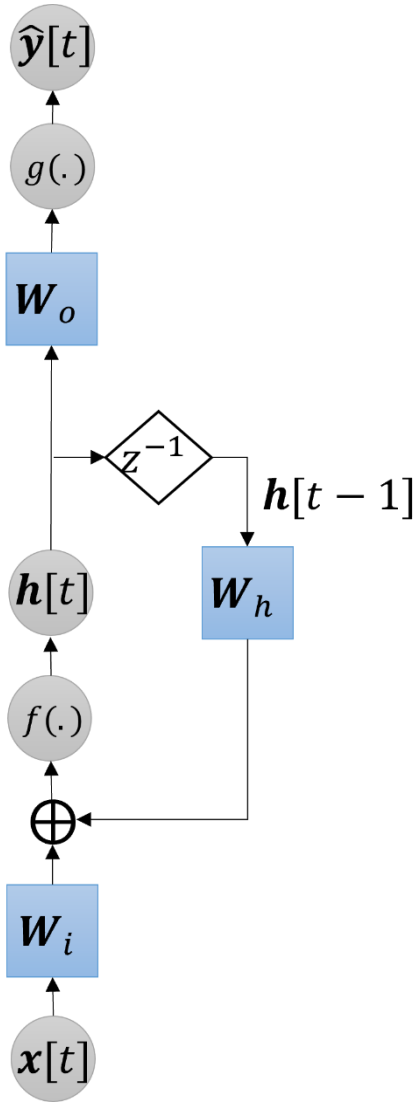
- As RNNs são compostas por **células (ou camadas) recorrentes.**
- A célula recebe a **entrada atual** $\boldsymbol{x}[t]$ e o **estado anterior** $\boldsymbol{h}[t-1]$.
- Ela combina os dois para calcular o **novo estado** $\boldsymbol{h}[t]$ e a **saída** $\hat{\boldsymbol{y}}[t]$.
- O **laço de realimentação** (em vermelho) é o que dá memória à rede.

Célula recorrente



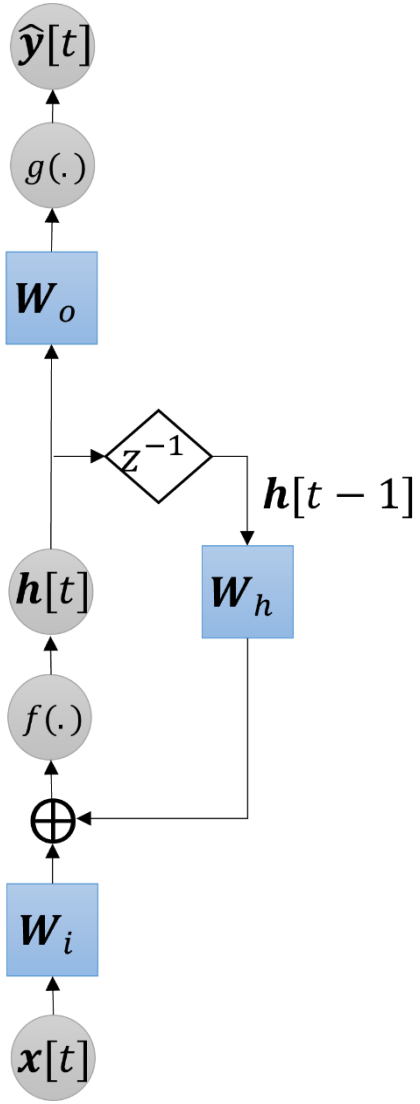
- $\boldsymbol{x}[t]$: vetor de entrada
- $\hat{\boldsymbol{y}}[t]$: vetor de saída
- $\boldsymbol{h}[t]$: vetor de estado interno ou oculto
- $\boldsymbol{h}[t-1]$: vetor de estado interno anterior
- $\boldsymbol{W}_i$: matriz de pesos de entrada
- $\boldsymbol{W}_h$: matriz de pesos da camada recorrente
- $\boldsymbol{W}_o$: matriz de pesos de saída
- z^{-1}: unidade de retardo de tempo
- $f(.)$ e $g(.)$: funções de ativação, e.g., *tanh* e *softmax*
- $\oplus$: soma de vetores

Célula recorrente



- A cada passo, a célula combina a **entrada atual** $\boldsymbol{x}[t]$ com o **estado oculto anterior** $\boldsymbol{h}[t-1]$.
- O resultado é o **novo estado oculto** $\boldsymbol{h}[t-1]$: um **resumo** da sequência até agora (i.e., a **memória**).
- A partir de $\boldsymbol{h}[t]$, a célula gera a **saída** $\hat{\boldsymbol{y}}[t]$.
- No treinamento, ela aprende **o que guardar do passado** e **como usá-lo** junto com a entrada.

Atualização do estado e geração da saída



- Equações de **atualização do estado oculto** e de **saída da célula** recorrente:

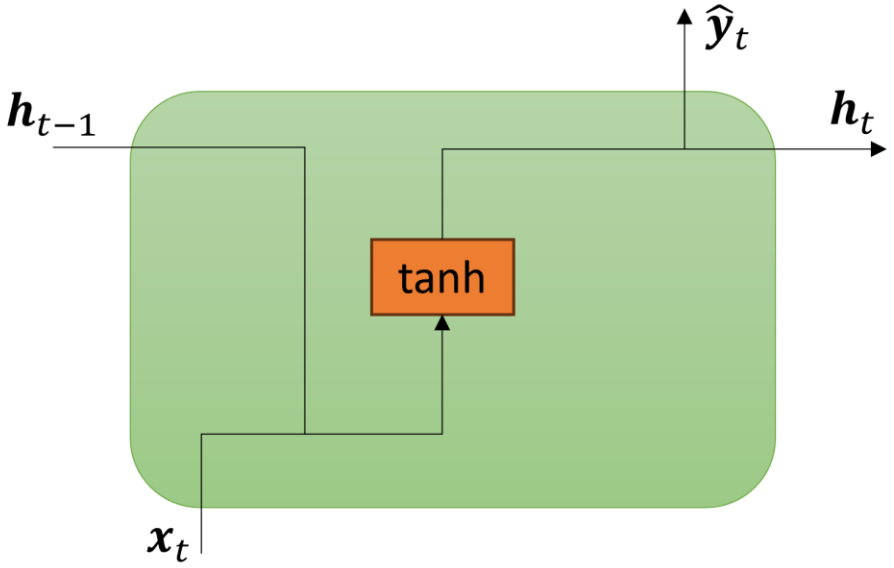
$$\boldsymbol{h}[t] = f(\boldsymbol{W}_h \boldsymbol{h}[t-1] + \boldsymbol{W}_i \boldsymbol{x}[t] + \boldsymbol{b}_h),$$

$$\hat{\boldsymbol{y}}[t] = g(\boldsymbol{W}_o \boldsymbol{h}[t] + \boldsymbol{b}_o),$$

onde $\boldsymbol{b}_h$ e $\boldsymbol{b}_o$ são vetores de *bias*.

- Os **pesos são os mesmos** em **todos os intervalos de tempo**.
- Em geral, usa-se funções de ativação, $f(.)$, do tipo **tangente hiperbólica** (tanh) ou **sigmóide**.
  - Não se usa ReLU, pois **por não saturarem**, a **recorrência** pode **causar explosão do gradiente**.

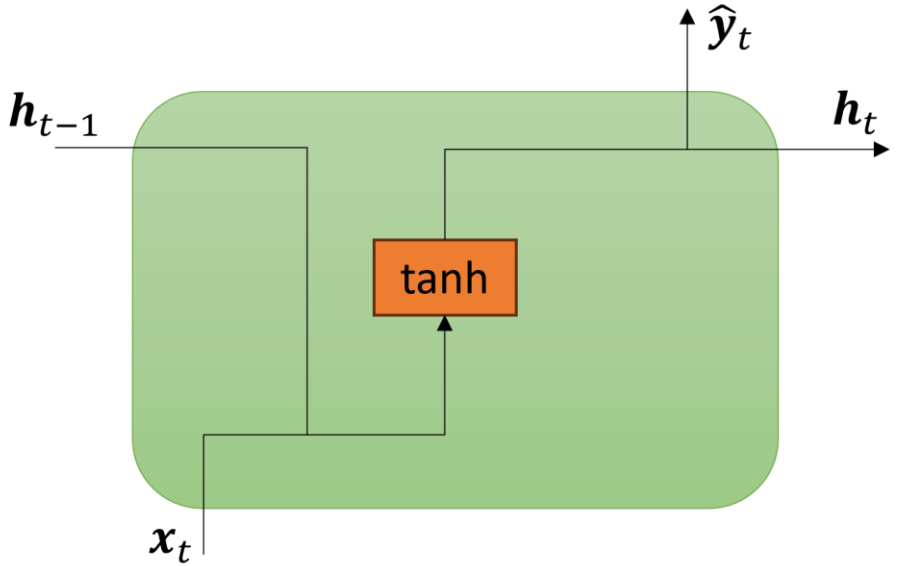
Célula recorrente



- Representação **simplificada** da célula recorrente.
- Ela omite a maioria das matrizes de pesos, mostrando uma caixa laranja que corresponde a uma camada densa com função de ativação do tipo tangente hiperbólica (*tanh*).
  - Ou seja, uma matriz de pesos com função *tanh*.
- Essa representação é bastante utilizada na literatura.

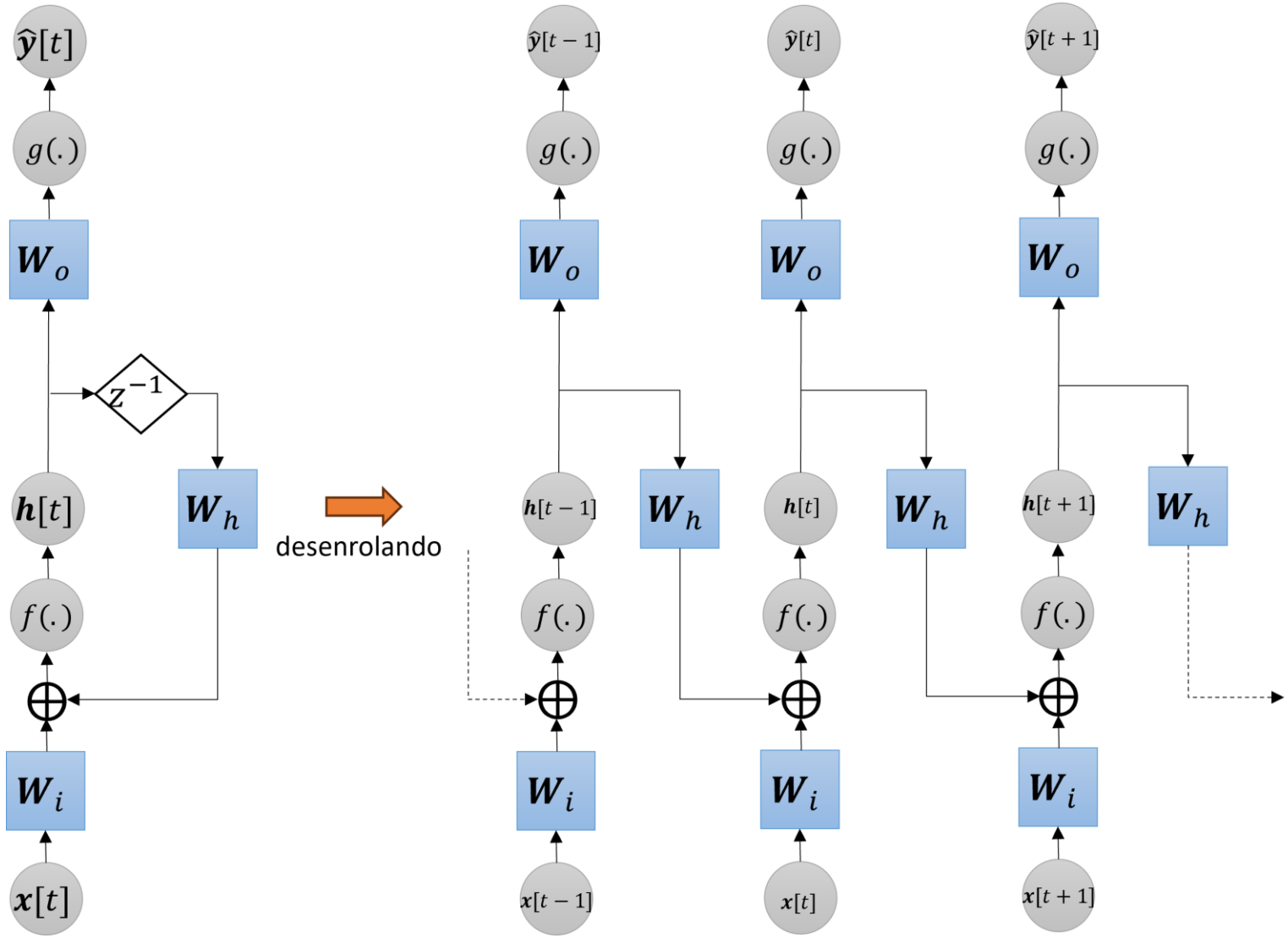
OBS.: Para facilitar a notação, usamos x_t ao invés de $x[t]$.

Célula recorrente: passo a passo



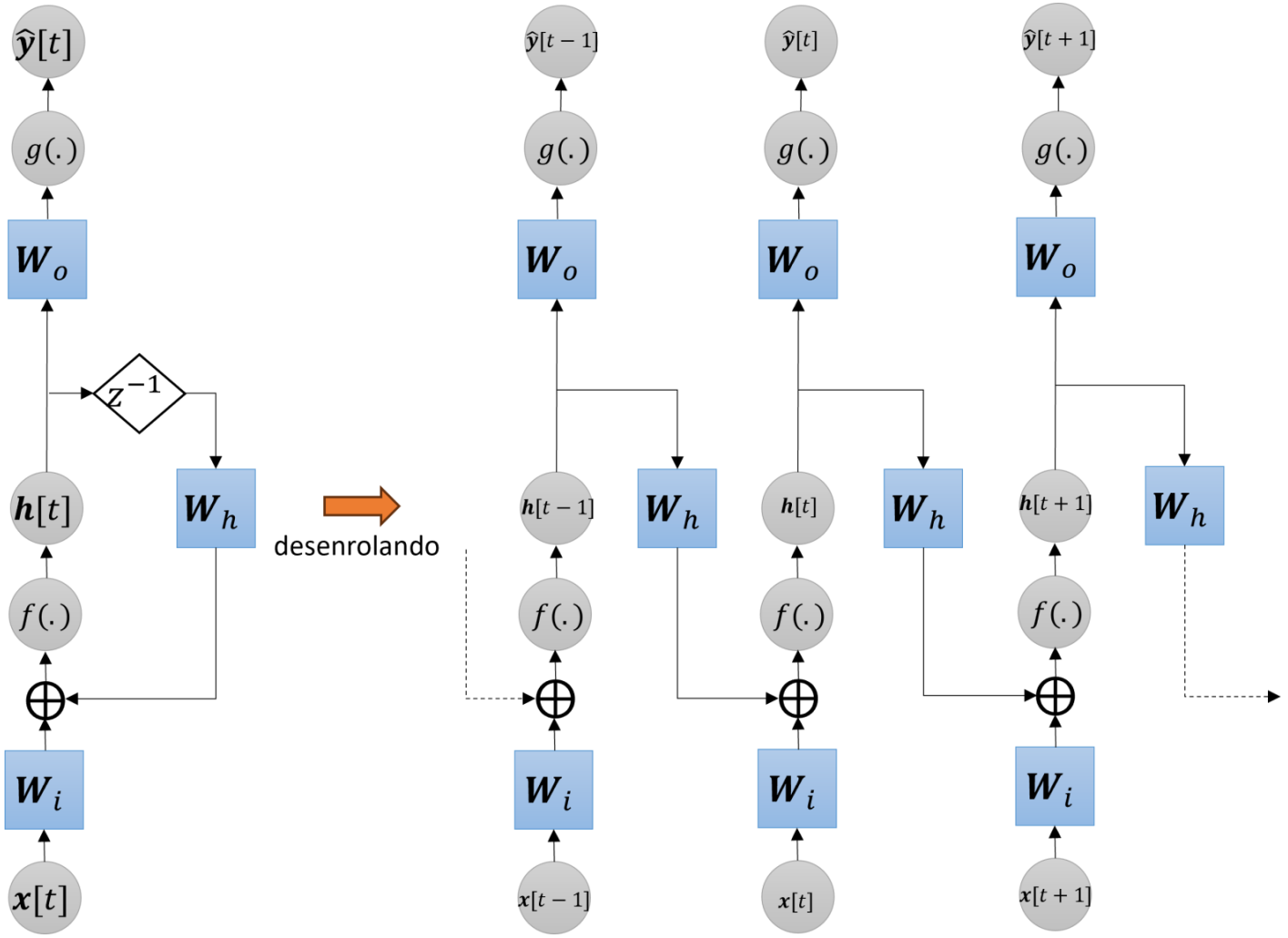
- A cada passo de tempo t, a célula lê **um elemento** da sequência (e.g., uma palavra) e faz:
  1. Recebe a entrada atual $\boldsymbol{x}_t$ e o estado oculto anterior $\boldsymbol{h}_{t-1}$ (a memória).
  2. Combina os dois e aplica *tanh*, obtendo o novo estado oculto $\boldsymbol{h}_t$.
  3. Gera a saída $\hat{\boldsymbol{y}}_t$ a partir de $\boldsymbol{h}_t$.
  4. Passa $\boldsymbol{h}_t$ como memória para o passo seguinte, $t + 1$.

Desdobrando uma RNN



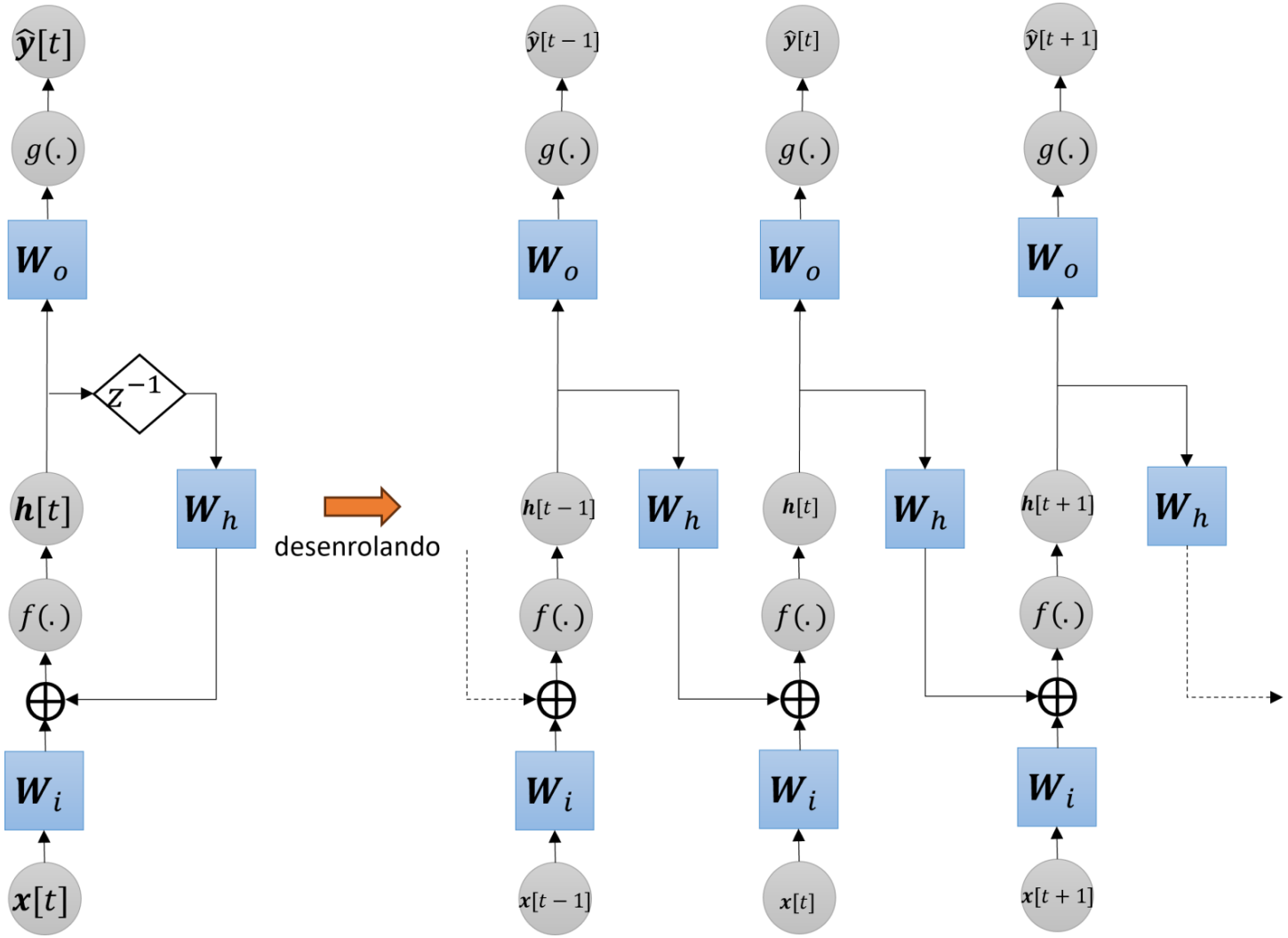
- O **laço** é uma forma compacta de desenhar a RNN.
- Para ver o que acontece ao longo da sequência, **desdobramos** a rede **no tempo**.
- Ela vira uma **cadeia de cópias da mesma célula**, uma por passo de tempo.
  - Todas as cópias usam **os mesmos pesos e ativações**.
- Cada cópia passa o **estado oculto** para a seguinte.

Desdobrando uma RNN



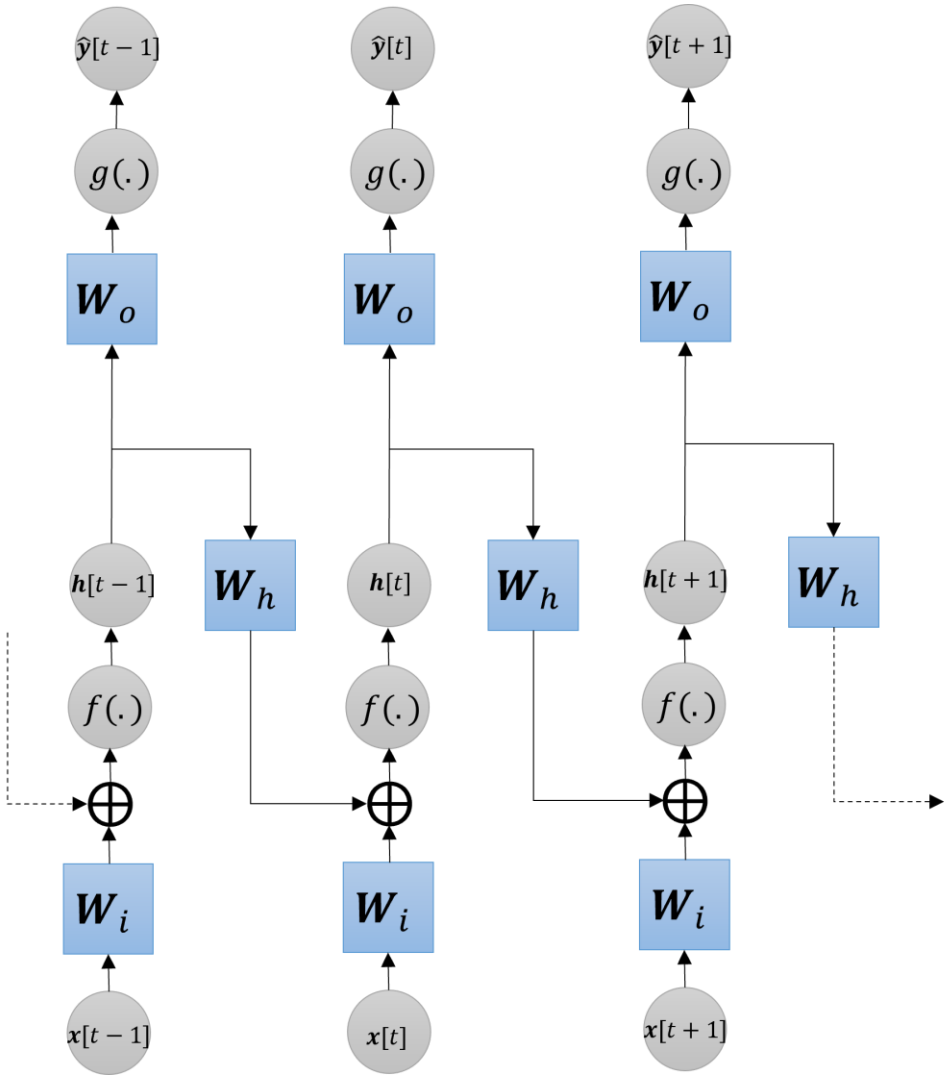
- Cada cópia da célula corresponde a **um passo de tempo** e funciona como uma **camada de uma rede profunda**, mas com **pesos compartilhados**.
- A **profundidade** da rede desdobrada é igual ao **comprimento da sequência**:
  - Uma frase de 50 palavras gera uma rede com 50 camadas.
- **A rede** se torna **muito profunda** para **sequências muito longas**.

Desdobrando uma RNN



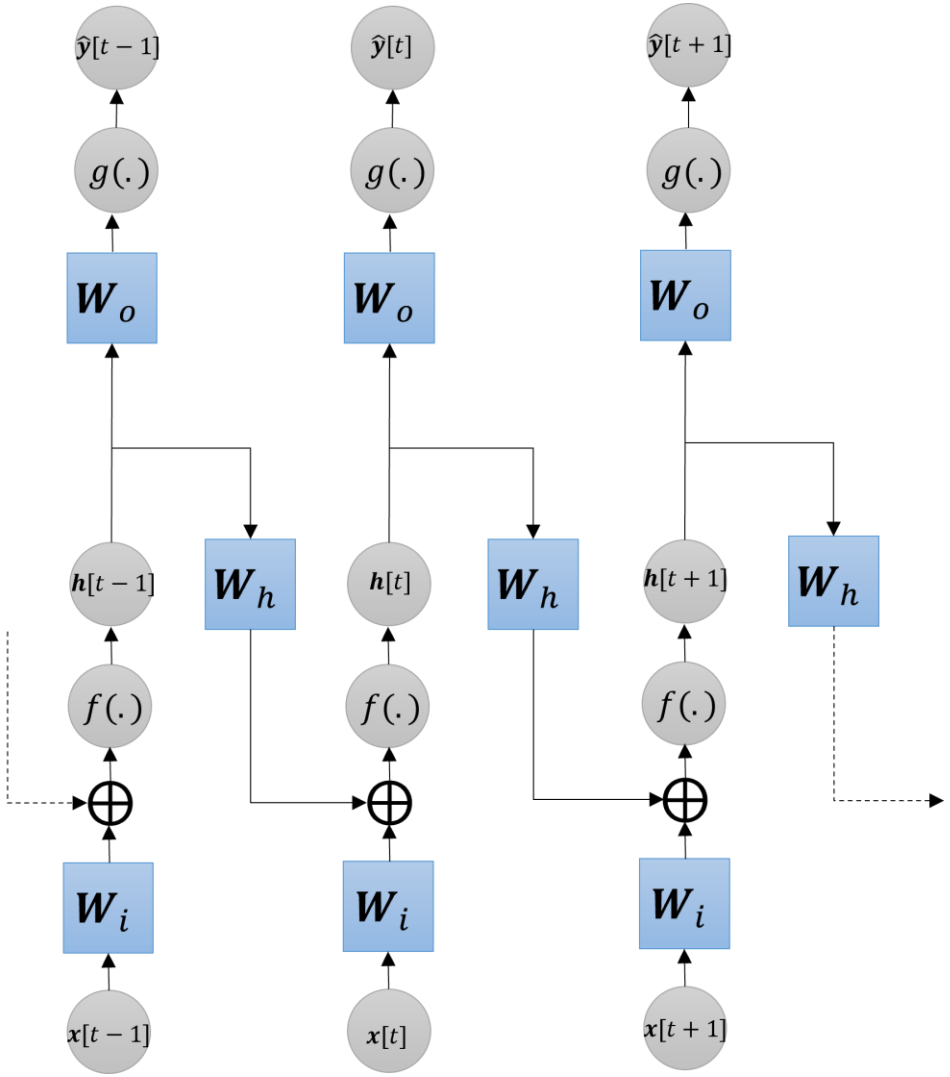
- Todas as cópias usam **os mesmos pesos**, então a rede **aplica o mesmo padrão em qualquer posição** da sequência.
- A saída $\hat{y}[t]$ depende de $x[t]$ **e de todas as entradas anteriores**, via estado oculto.
- Em uma MLP, a saída depende só de $x[t]$.

Desdobrando uma RNN



- O processo de desdobrar uma RNN possibilita o treinamento da rede usando a **retropropagação do erro**.
- O objetivo é:
  - descobrir como cada peso influencia o erro,
  - ajustar os pesos para minimizar o erro (e.g., EQM, EC).
- Nas RNNs, isso é mais complexo pois:
  - o estado atual depende dos estados anteriores,
  - então o **erro precisa voltar no tempo**.

Treinando uma RNN



- A RNN é **desdobrada no tempo** e treinada como uma rede profunda comum, usando a **retropropagação do erro**.
- Inicialmente, a sequência de T passos, $(x_1, x_2, \ldots, x_T)$, é processada pela RNN em uma **etapa direta**, gerando saídas para os T passos, $(\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T)$.
- Em seguida, o **erro de cada passo**, E_t, é calculado:

$$E_t = e(y_t, \hat{y}_t),$$

onde y_t são os rótulos de cada passo.

Treinando uma RNN

- Na sequência, o **erro total**, E, (**média temporal dos erros de cada passo**) é calculado

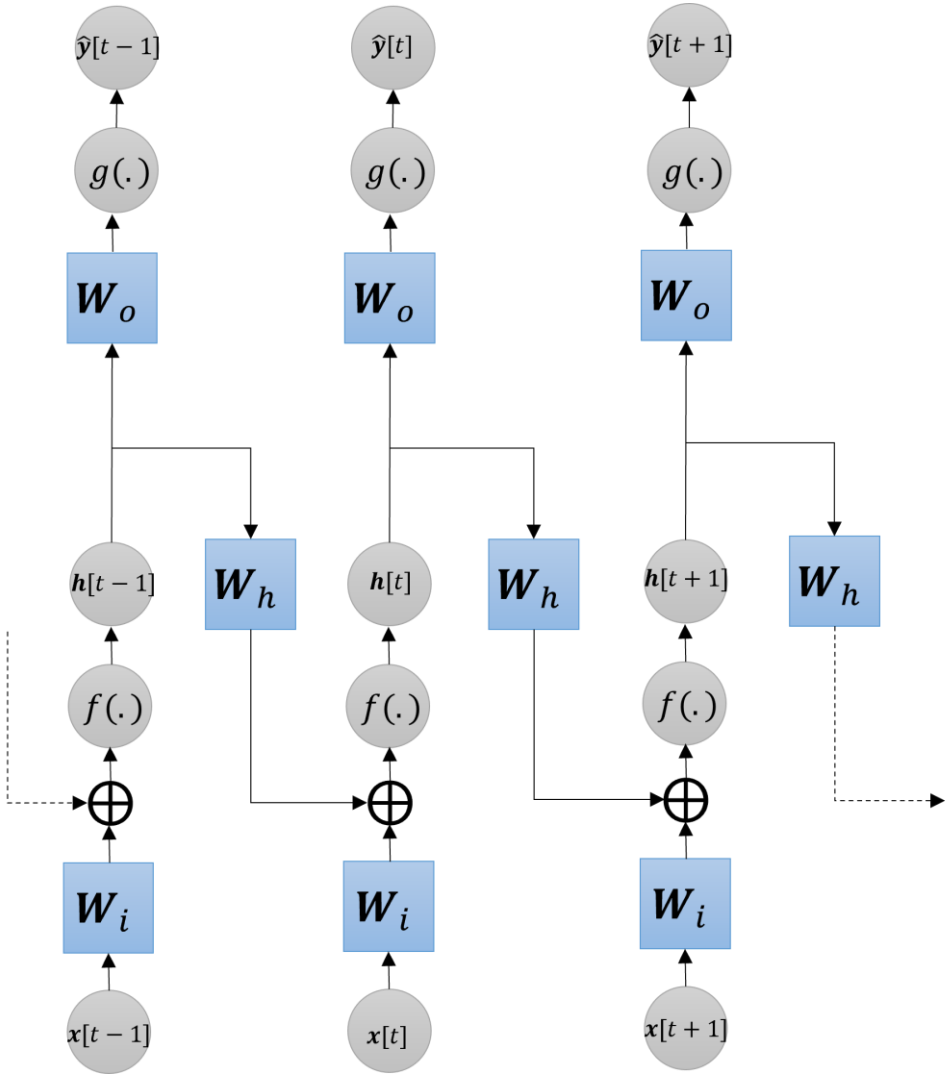
$$E = \frac{1}{T}\sum_{t=0}^{T-1} E_t = \frac{1}{T}\sum_{t=0}^{T-1} e(y[t], \hat{y}[t])\,.$$

- Em seguida, propaga-se o gradiente de $t = T - 1$ até $t = 0$ (do último passo ao primeiro), através da rede desdobrada.
- Após, acumula-se os gradientes. Como os **pesos são compartilhados**, o gradiente de cada matriz é a **soma das contribuições de todos os passos**:

$$\frac{\partial E}{\partial \boldsymbol{W}} = \sum_{t=0}^{T-1} \left(\frac{\partial E}{\partial \boldsymbol{W}}\right)_t,$$

onde $\boldsymbol{W}$ pode ser qualquer uma das matrizes de peso da célula.

Treinando uma RNN



- Por fim, com o gradiente total, as matrizes são atualizadas com um método de otimização (e.g., GDB, SGD, Adam).
- Por exemplo, usando o SGD temos

$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \alpha \frac{\partial E}{\partial \boldsymbol{W}},$$

onde α é a taxa de aprendizagem.

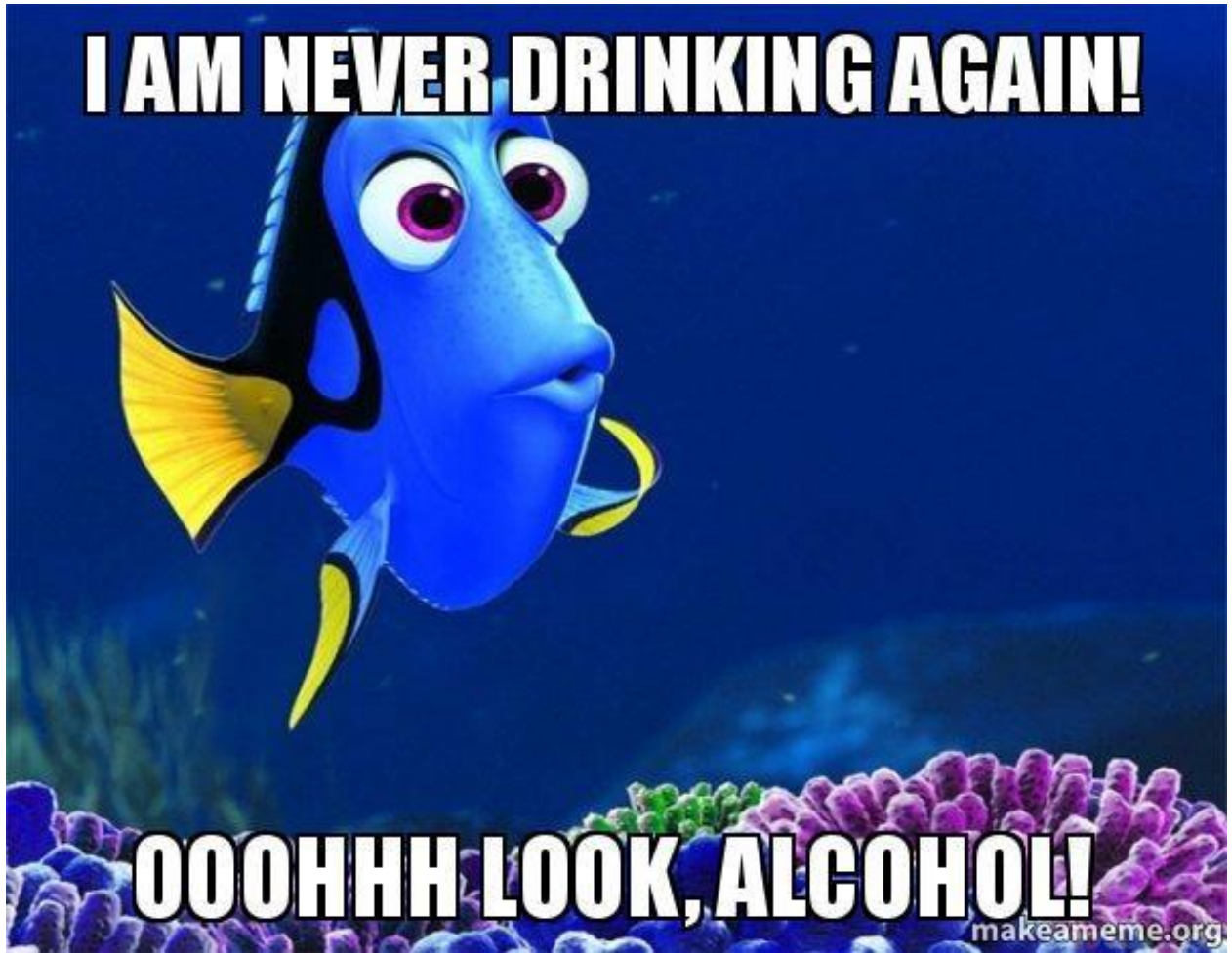
- Após a atualização, volta-se à etapa direta com uma nova sequência (ou conjunto).
  - Usamos uma sequência, mas para o gradiente ficar mais estável, usa-se um conjunto de sequências (i.e., mini-lote).

Treinando uma RNN: BPTT

- O algoritmo que aplica a retropropagação à rede desdobrada é chamado de *Backpropagation Through Time* (BPTT).
- É *backpropagation* tradicional, mas como os **pesos são compartilhados**, então os **gradientes dos passos devem ser somados**.
- A ideia central é:

  o **erro atual** pode ter **sido causado por estados passados**.

- Então o gradiente precisa "**voltar no tempo**" e atualizar os pesos.
- **Exemplo:** "*O gato subiu no* ___", o estado de "gato" ajuda a prever "muro" lá na frente. Se a rede erra em "muro", parte da responsabilidade volta até o estado de "gato".



Quando uma **RNN tradicional** processa uma **sequência muito longa**, ela se **esquece gradualmente das primeiras amostras** da sequência.

Ou seja, elas **não aprendem dependências longas...**

Vamos entender o motivo.

Backpropagation Through Time (BPTT)

- De forma genérica, a derivada do erro total em relação a matriz de pesos, $\boldsymbol{W}$, é obtida usando-se a **regra da cadeia**:

$$\frac{\partial E}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial E_t}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial e(y[t],\hat{y}[t])}{\partial \hat{y}[t]}\frac{\partial \hat{y}[t]}{\partial h[t]}\frac{\partial h[t]}{\partial \boldsymbol{W}}.$$

onde $h[t] = f(x[t], h[t-1], \boldsymbol{W})$ é o **estado oculto**.

- Como os pesos são compartilhados ao longo do tempo, uma única atualização de matriz deve levar em conta sua influência na predição (e, consequentemente, no erro) feita em $t = 0$ até $t = T - 1$.
- Em outras palavras, o **BPTT propaga os erros do futuro para o passado**.

Backpropagation Through Time (BPTT)

$$\frac{\partial E}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial E_t}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial e(y[t],\hat{y}[t])}{\partial \hat{y}[t]}\frac{\partial \hat{y}[t]}{\partial h[t]}\boxed{\frac{\partial h[t]}{\partial \boldsymbol{W}}}.$$

- Os dois primeiros termos são fáceis de se calcular, mas o terceiro é onde as coisas se complicam, já que precisamos calcular **recursivamente** o efeito da matriz de pesos $\boldsymbol{W}$ sobre $h[t]$.
- $\boldsymbol{W}$ afeta o erro E_t de duas maneiras:
  - **Diretamente:** No cálculo do estado oculto atual $h[t]$.
  - **Indiretamente:** Porque $h[t]$ depende de $h[t-1]$, que por sua vez dependeu de $\boldsymbol{W}$ no passado, que dependeu de $h[t-2]$, e assim por diante, voltando até o estado inicial $h[0]$.

Backpropagation Through Time (BPTT)

- Aplicando a regra da cadeia ao terceiro termo, obtemos:

$$\frac{\partial E}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial E_t}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=0}^{t}\frac{\partial E_t}{\partial \hat{y}[t]}\frac{\partial \hat{y}[t]}{\partial h[t]}\left(\prod_{j=k+1}^{t}\frac{\partial h[j]}{\partial h[j-1]}\right)\frac{\partial^{+} h[k]}{\partial \boldsymbol{W}}.$$

- O gradiente envolve o **produtório de muitas derivadas (Jacobianos)**.
- O **produtório** é o caminho que o **gradiente** percorre de volta de t até k, com $t - k$ fatores.
- O **produtório pode causar o desaparecimento ou a explosão do gradiente** dependendo dos valores das derivadas parciais, $\frac{\partial h[j]}{\partial h[j-1]}$.

*∂^{+} é o efeito direto de $\boldsymbol{W}$ no passo k tratando $h[k-1]$ como constante

Desaparecimento e explosão do gradiente

- Se $\frac{\partial h[j]}{\partial h[j-1]} \forall j < 1$, então o produto tende a **zero**
  - A rede **esquece o passado mais distante** (informação antiga se perde).
  - Não aprende **dependências de longo prazo, apenas curtas.**
    - Estados ocultos anteriores não têm efeito sobre os estados posteriores.
  - Treinamento se torna **muito lento** ou fica **estagnado**.
- Se $\frac{\partial h[j]}{\partial h[j-1]} \forall j > 1$, então o produto **explode**
  - O modelo **diverge**.
  - Atualizações de pesos se tornam enormes.
  - **Treinamento instável**: erro vira NaN.
  - Sintoma típico: **oscilação do erro**.
- Na prática, o desaparecimento do gradiente é mais comum.

Como mitigar esses problemas?

- **Para a explosão**:
  - Limitar a magnitude do gradiente (*gradient clipping*)
- **Para o desaparecimento**:
  - Usar arquiteturas de RNNs mais avançadas: LSTM e GRU
    - Usam **portões**, que funcionam como **válvulas que controlam o fluxo da informação e criam um caminho com derivada ≈ 1, o que mantém o gradiente estável por vários passos.**
  - **Conexões residuais**: criam **caminhos diretos** para o gradiente entre camadas empilhadas
- **Para ambos**:
  - Inicializar os pesos de forma apropriada (inicializações de He, Xavier e ortogonal)
  - BPTT truncado.

Backpropagation Through Time (BPTT)

$$\frac{\partial E}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial E_t}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=0}^{t}\frac{\partial E_t}{\partial \hat{y}[t]}\frac{\partial \hat{y}[t]}{\partial h[t]}\left(\prod_{j=k+1}^{t}\frac{\partial h[j]}{\partial h[j-1]}\right)\frac{\partial^{+} h[k]}{\partial \boldsymbol{W}}.$$

- Se o valor de T for muito grande (i.e., uma **sequência muito longa**), a rede desdobrada fica **muito profunda**:
  - o que torna o **treinamento muito lento** (muitos passos em sequência),
  - **consome muita memória** (guardar todos os estados),
  - e pode deixar o **treinamento instável devido à recursão** (produto de muitos Jacobianos).
- Podemos **propagar o erro até um determinado passo, $\tau > 0$**, ao invés de **propaga-lo até o início da sequência.**

Truncated Backpropagation Through Time (TBPTT)

- O processo de limitar a propagação do erro até τ passos é chamado de **BPTT truncado (TBPTT)**.
  - τ é um hiperparâmetro que deve ser otimizado: pequeno demais prejudica a memória de longo prazo e grande demais perde a eficiência que motivou a técnica.
- Ele reduz:
  - custo computacional,
  - uso de memória,
  - e ameniza a explosão e o desaparecimento do gradiente.
- Porém, o TBPTT força a rede a se concentrar em padrões de curto e médio prazo.
- Se a informação relevante está a mais de τ passos, o TBPTT não consegue ensinar a rede a usá-la.

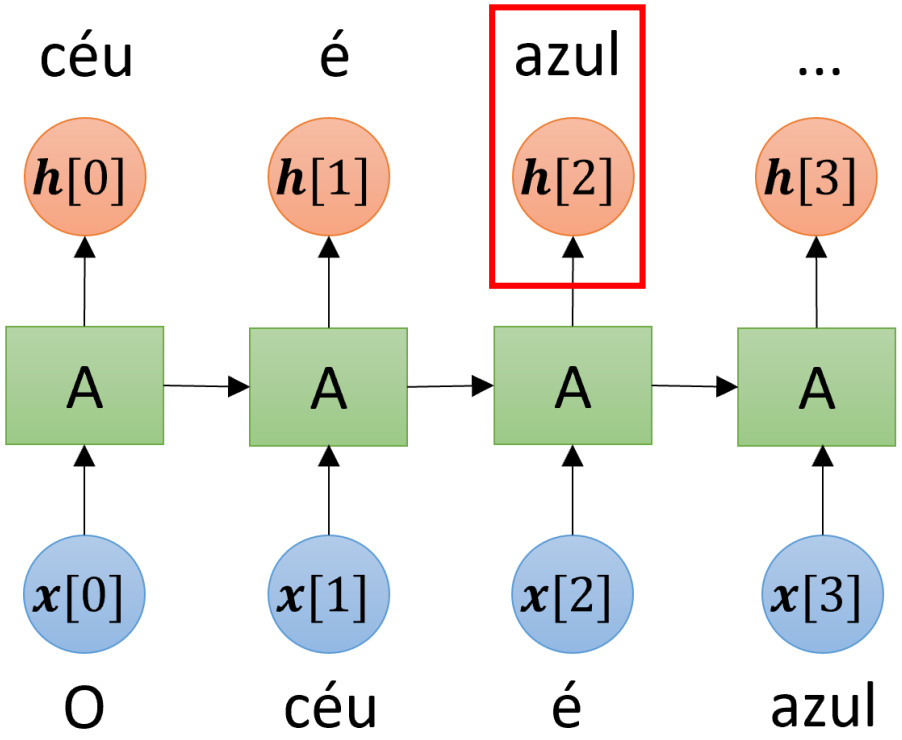
Por que o BPTT truncado funciona?

- Em muitos problemas, o que importa para prever o próximo elemento está **perto**.
  - A influência do passado **diminui com a distância**.
- As contribuições de passos distantes já são **pequenas** devido ao produto de muitos Jacobianos.
  - Cortar o gradiente em τ passos **perde pouca informação**.
- Ou seja, **nem sempre é necessário voltar até o primeiro passo** da sequência.
- **OBS.:** na prática, o estado oculto continua sendo passado adiante entre os blocos, e só o gradiente é cortado.

O problema das dependências de longo prazo

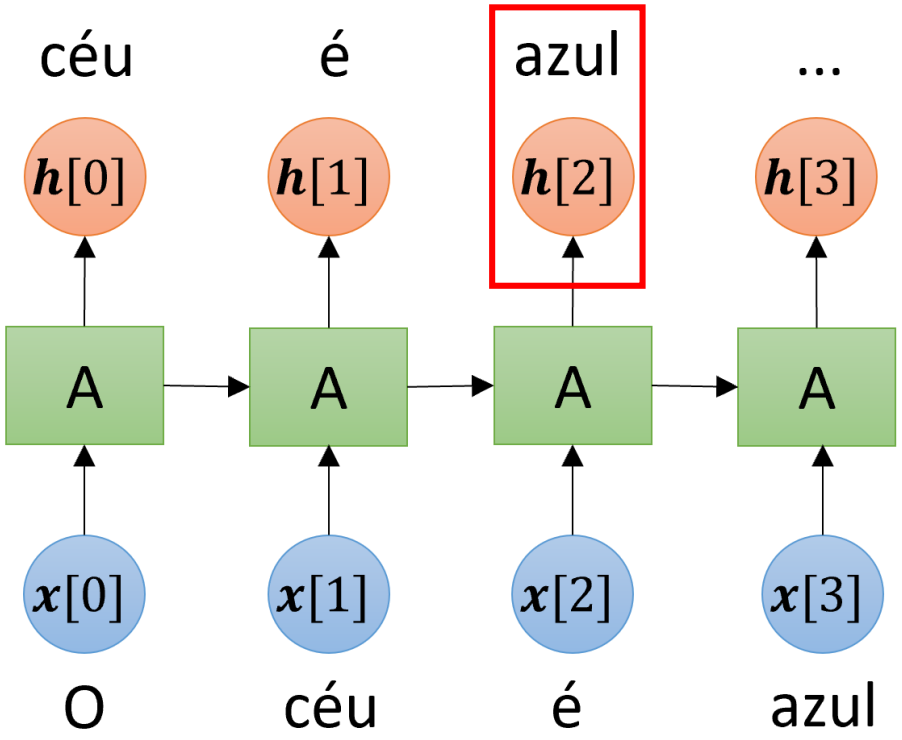
- **RNNs simples** (baunilha) não aprendem dependências de longo prazo: o gradiente **desaparece** ao voltar no tempo.
- O **TBPTT** também limita esse aprendizado, pois corta o gradiente após τ passos.
- Porém, muitas tarefas exigem contexto **curto e longo**.
  - **Exemplo**: na tradução de texto, uma palavra pode depender das vizinhas ou de algo dito muito antes.

O problema das dependências de longo prazo



- Considere treinar a RNN ao lado para prever **a próxima palavra**.
- Para **prever** a palavra **"azul"**, a RNN precisa apenas de **informação de dois passos atrás**.
- Ou seja, a **informação relevante está a poucos passos** no tempo (**contexto recente**).

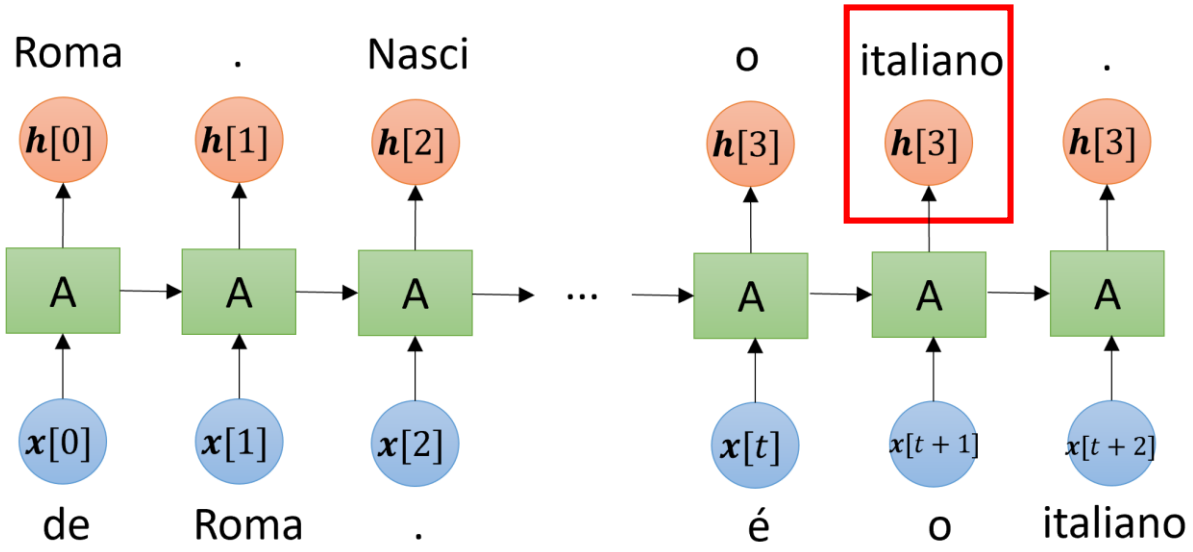
O problema das dependências de longo prazo



- Assim, durante o **treinamento**, o **erro precisa ser retropropagado por apenas alguns passos**.
- Porém, em muitas aplicações, a **informação importante pode estar muito distante na sequência**.
- Consideremos o seguinte trecho:

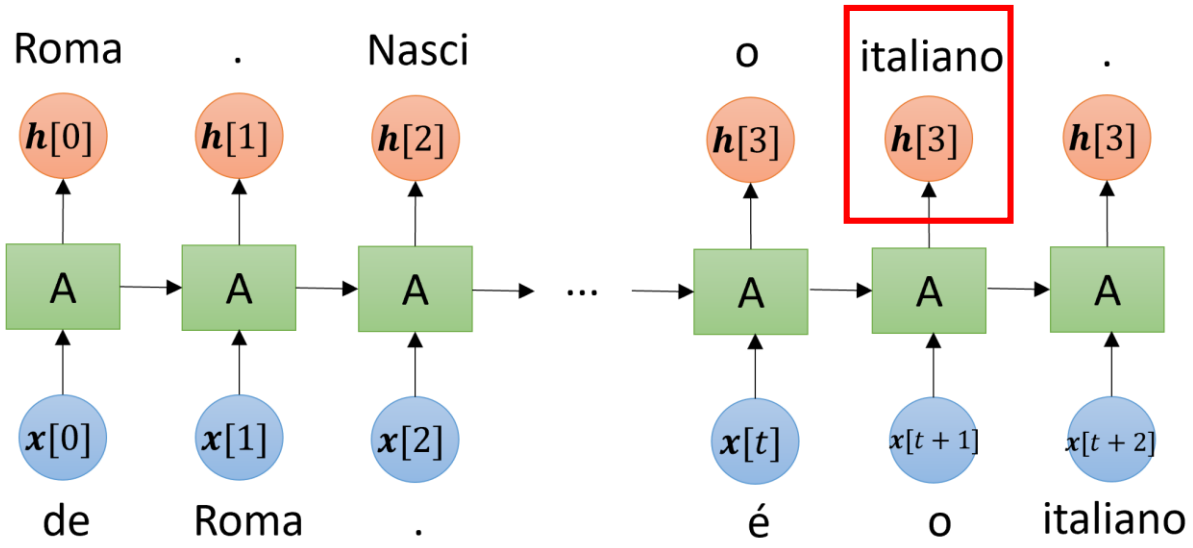
  "Sou de **Roma**. Nasci há 30 anos. Gosto de comer bem e andar de bicicleta. Minha língua materna é o **italiano**."

O problema das dependências de longo prazo



- Para prever **"italiano"**, a rede precisa lembrar de uma **informação muito antiga** da sequência, a palavra **"Roma"**.
- Durante o treinamento, o **erro** precisa ser **retropropagado** por **vários instantes de tempo**, o que pode fazer com que o **gradiente desapareça**.

O problema das dependências de longo prazo



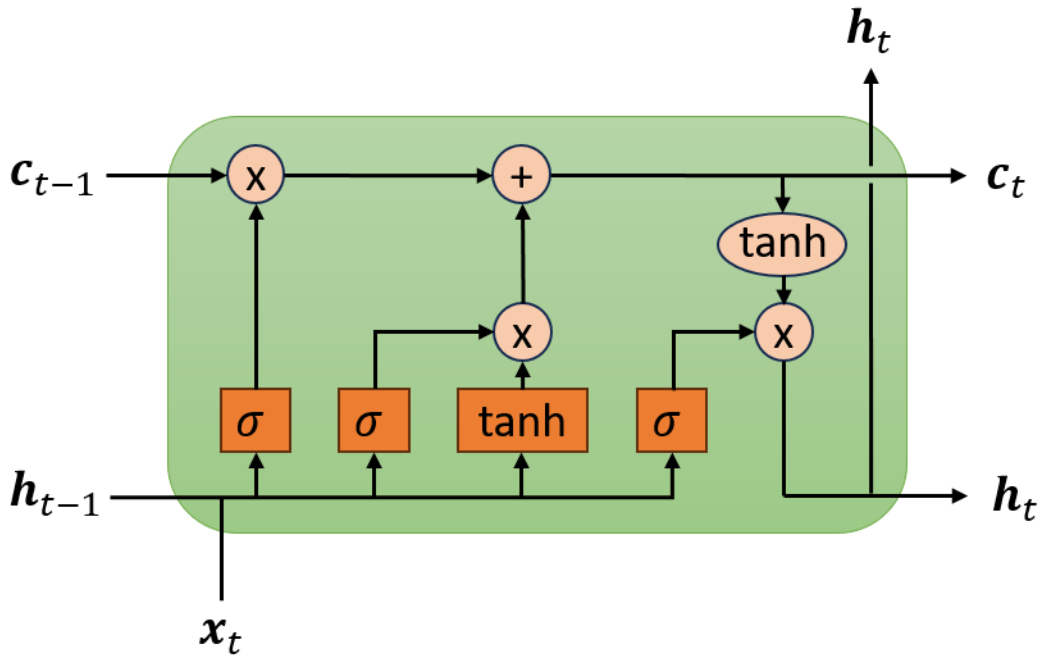
- **Quanto maior a distância temporal**, **mais difícil é para uma RNN tradicional** manter a informação relevante, ou seja, **se lembrar do passado distante**.

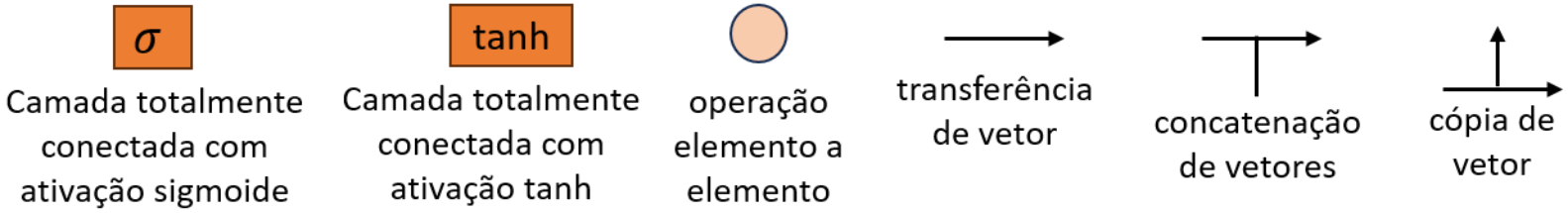
Como permitir que a rede preserve **informações importantes por mais tempo**?

O problema das dependências de longo prazo

- Para **diminuir o problema das dependências de longo prazo**, causado pelo desaparecimento do gradiente, foram propostas **redes com memória controlada por portões:**
  - *Long Short-Term Memory* (LSTM)
  - *Gated Recurrent Unit* (GRU)
- Essas redes **criam caminhos ao longo do tempo** cujas **derivadas podem ser controladas** para não desaparecer.
- Os **portões** funcionam como válvulas que decidem:
  - o que **gravar** na memória,
  - o que **esquecer** ou **manter**,
  - o que **expor** na saída.

Long-short term memory (LSTM)

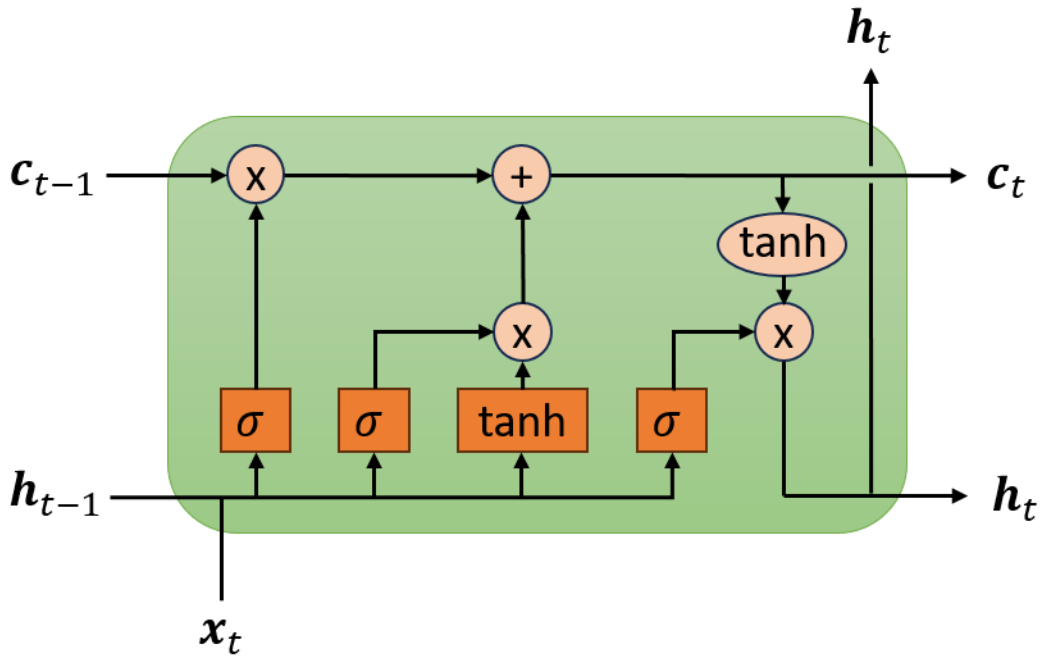


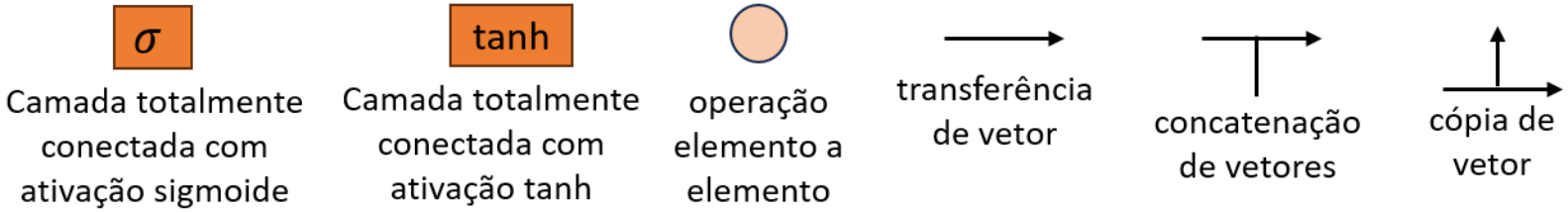


- Introduzida por Hochreiter e Schmidhuber em 1997.
- Funcionam muito bem em **diversos problemas que envolvem dados sequenciais**.
- São amplamente utilizadas atualmente.

OBS.: Para facilitar a notação, usamos x_t ao inves de $x[t]$.

Long-short term memory (LSTM)

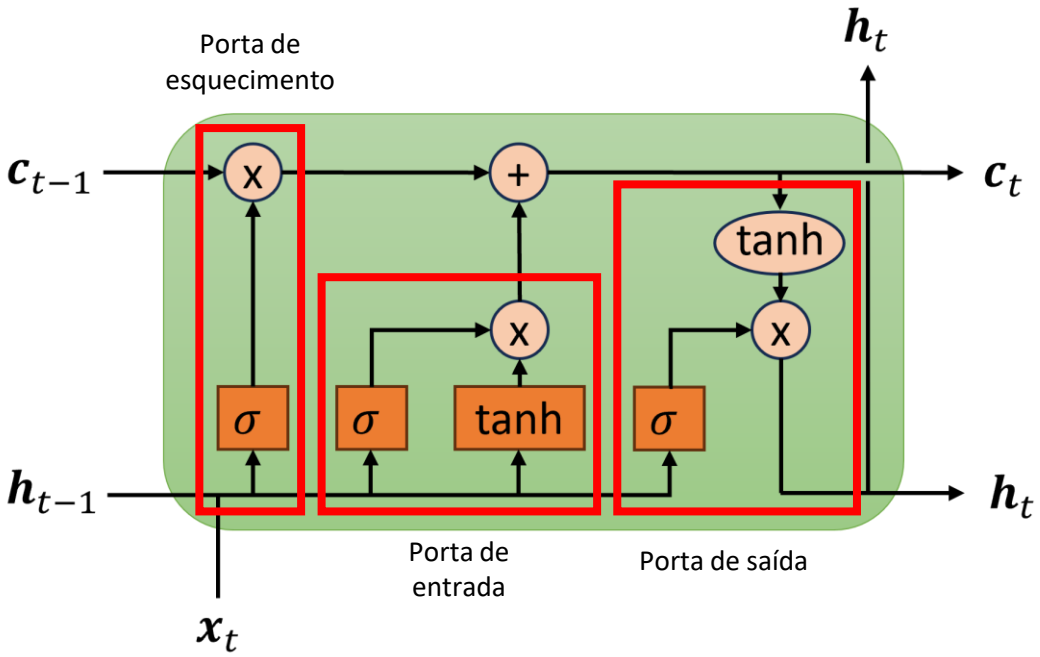




- Elas **capturam padrões de longo e curto prazo** de uma sequência de dados.
- Assim como as RNNs tradicionais, elas **precisam ser desdobradas** no tempo para serem **treinadas**.

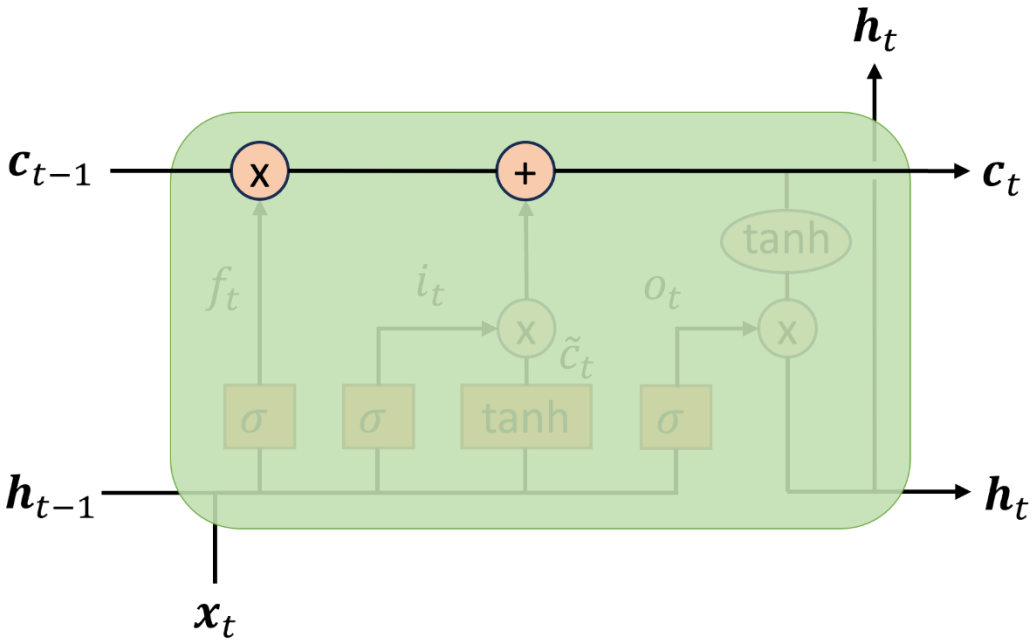
OBS.: Para facilitar a notação, usamos x_t ao inves de $x[t]$.

Long-short term memory (LSTM)



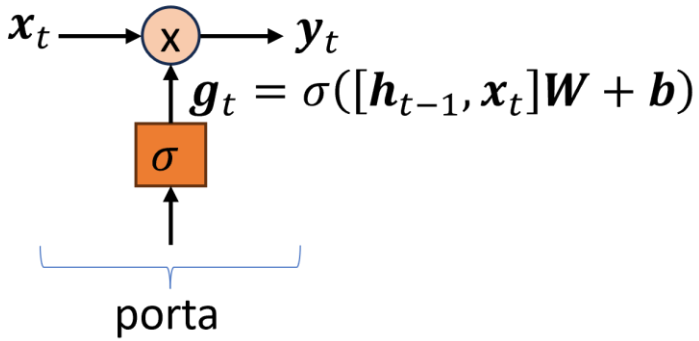
- A **unidade de processamento** (neurônio) da LSTM é mais **complexa** e é chamada de **célula (de memória)**.
- Uma célula LSTM é composta por **4 camadas totalmente conectadas**, que formam **3 portas**:
  - ***Forget gate*** (porta de esquecimento)
  - ***Input gate*** (porta de entrada)
  - ***Output gate*** (porta de saída)
- A informação na célula é **protegida e controlada** por meio destas 3 portas.

Long-short term memory (LSTM)



- Uma célula LSTM possui dois estados:
  - o estado interno da célula, $\boldsymbol{c_t}$
  - e o estado oculto da célula, $\boldsymbol{h_t}$.
- $\boldsymbol{c_t}$ é a memória de longo prazo.
- $\boldsymbol{h_t}$ é memória de curto prazo e a saída da célula.

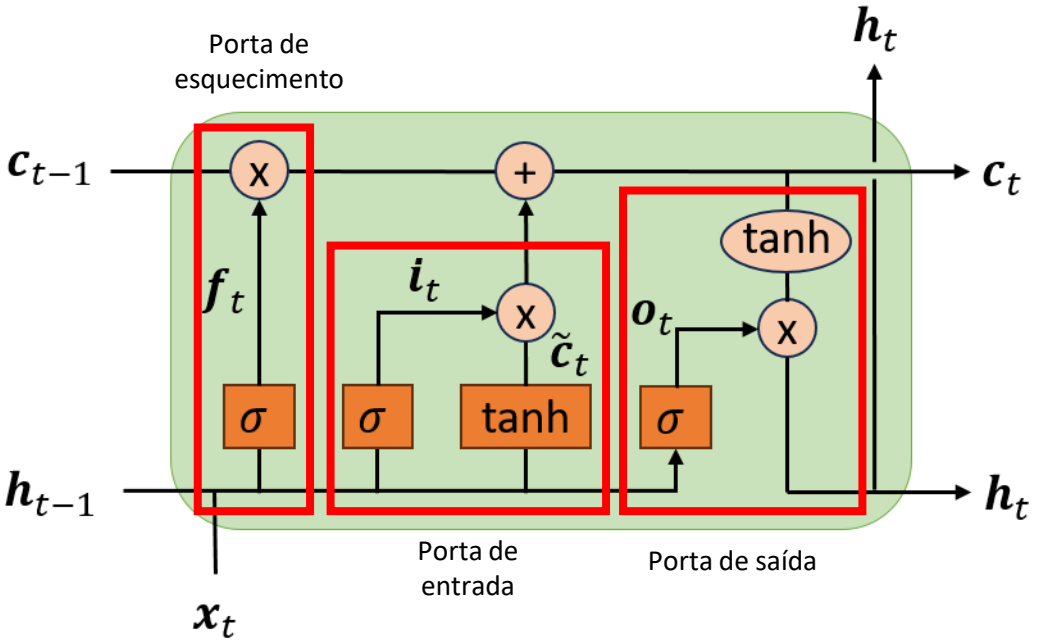
Como as portas funcionam?



x_t		g_t		y_t
0.58		0.2		0.116
-0.32	x	0.99	=	-0.3168
0.1		0.01		0.001
1.24		0.95		1.178

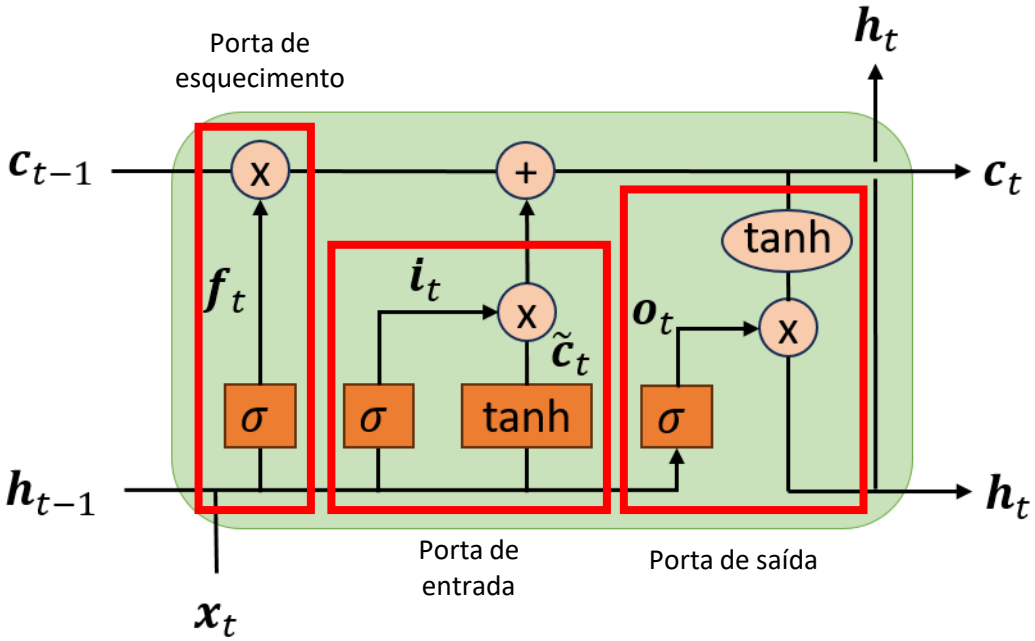
- As portas são uma forma de **permitir** a **passagem ou não de informações**.
- Imaginem uma das 3 camadas com função de ativação sigmoide da célula LSTM.
- A camada gera um **vetor**, g_t, com **elementos assumindo valores entre 0 e 1**.
- Uma **operação de multiplicação elemento a elemento** decide **o quanto da entrada**, x_t passa para a saída, y_t.
- A porta **aprende** (através dos **pesos**) o quanto de cada elemento deve ser permitido passar.

Long-short term memory (LSTM)



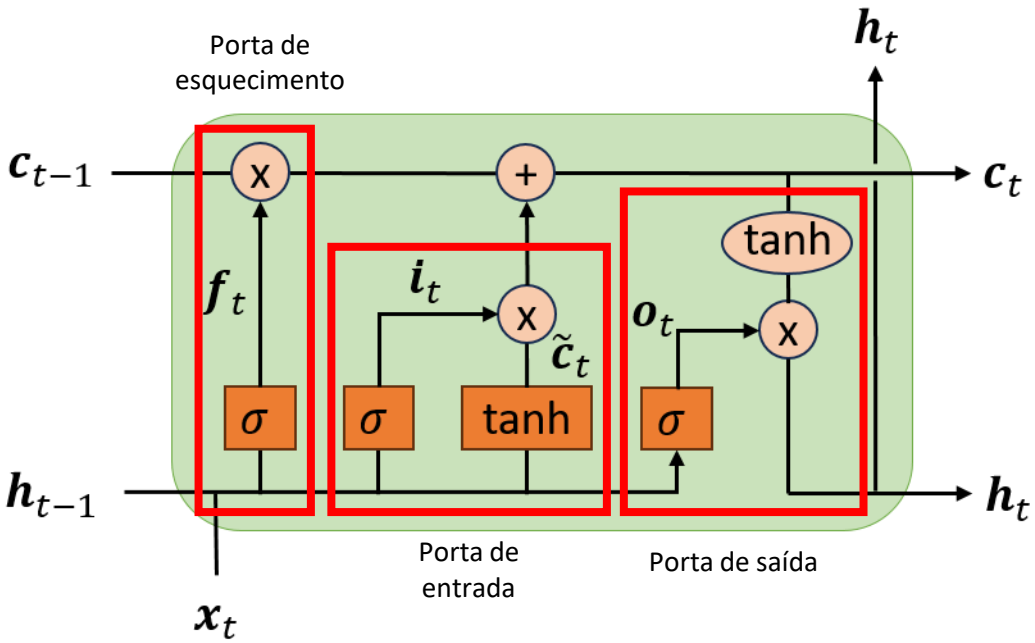
- A **porta de esquecimento** (controlada pelo vetor $\boldsymbol{f}_t$) controla **quais partes** da **informação passada**, $\boldsymbol{c}_{t-1}$, devem ser **apagadas ou mantidas**.
- A **porta de entrada** (controlada pelo vetor $\boldsymbol{i}_t$) controla **quais partes** de **novas informações (memórias)**, $\tilde{\boldsymbol{c}}_t$, devem ser **adicionadas** à $\boldsymbol{c}_{t-1}$.
- A **porta de saída** (controlada pelo vetor $\boldsymbol{o}_t$) controla **quais partes** de $\boldsymbol{c}_t$ devem ser **expostas na saída**, $\boldsymbol{h}_t$.

Como as portas funcionam?



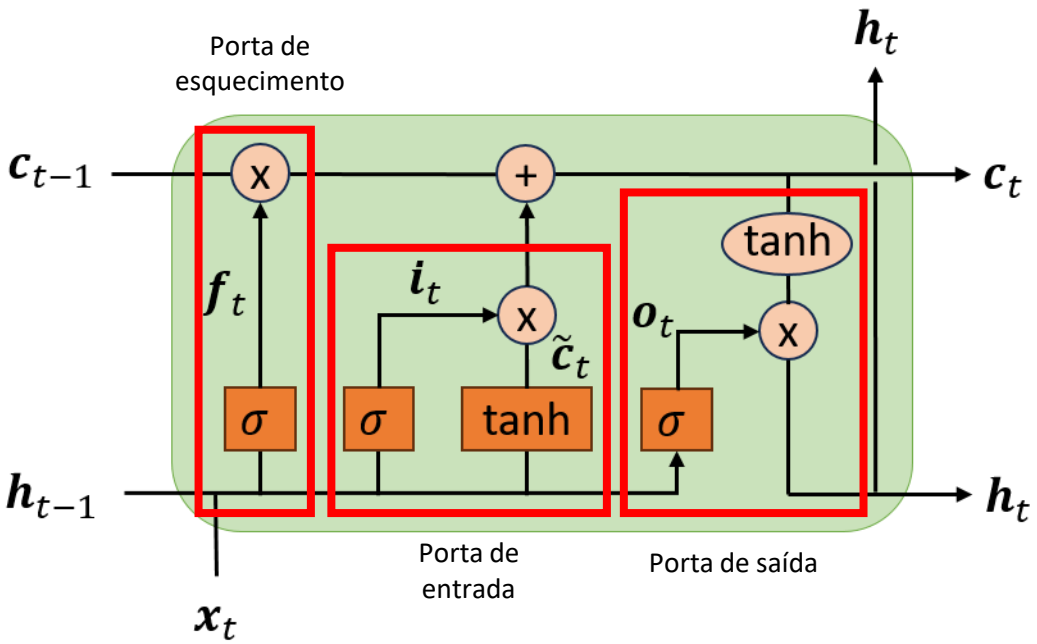
- **Porta de entrada**: uma **nova informação** é **adicionada** à **memória de longo prazo**, c_{t-1}, sempre que **um elemento de i_t tender a 1**.
- **Porta de esquecimento**: uma **informação de longo prazo** é **descartada** da **memória de longo prazo**, c_{t-1}, sempre que **um elemento de f_t tender a 0**.

Como as portas funcionam?



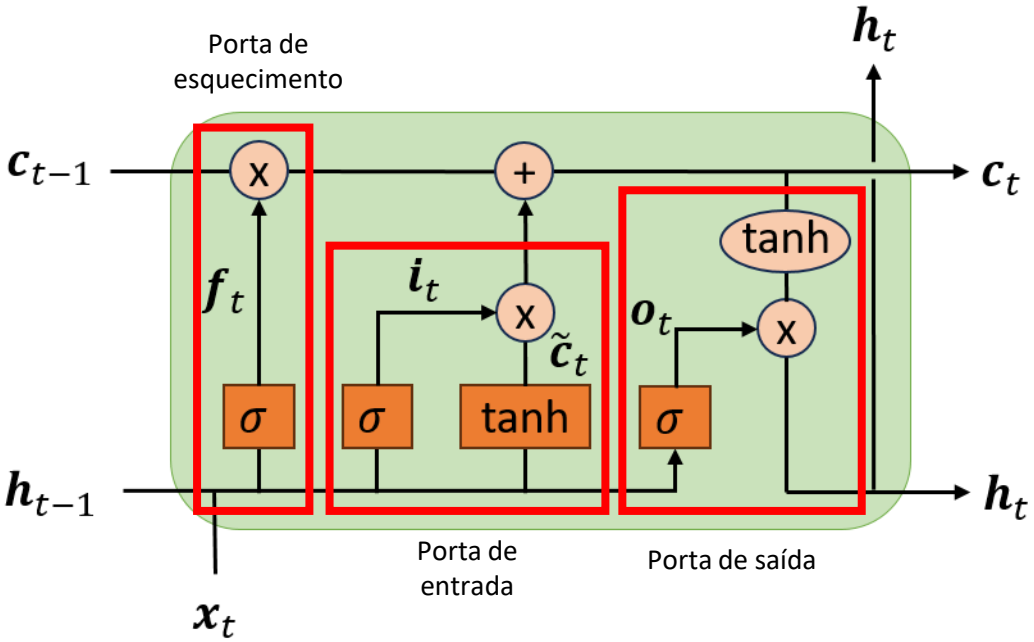
- **Porta de saída**: **uma informação de longo prazo** é **exposta** na **saída da célula** sempre que **um elemento de o_t tender a 1**.
- Em resumo, uma **célula** LSTM **aprende a reconhecer** uma **informação importante**, **armazená-la** no estado de longo prazo, **preservá-la** pelo tempo que for necessário e **expô-la** sempre que necessário.

Dinâmica das portas em uma célula LSTM



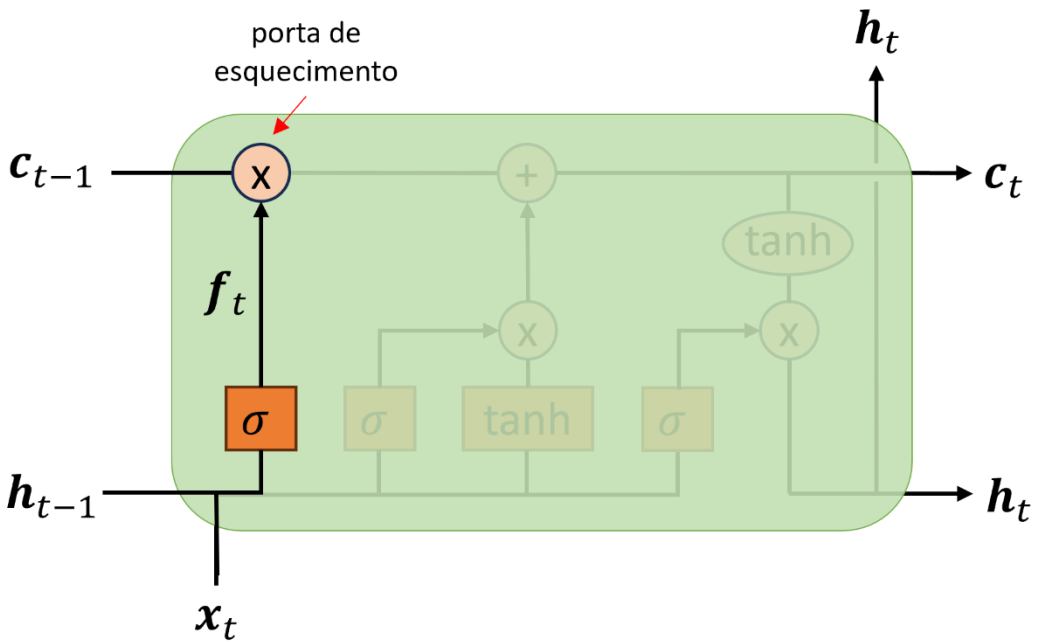
- Se $\boldsymbol{f}_t$ for sempre **1** e $\boldsymbol{i}_t$ for sempre **0**, então
$$\boldsymbol{c}_t = \boldsymbol{c}_{t-1}.$$
- Ou seja, o **estado interno da célula permanecerá constante para sempre**, passando inalterado para cada passo de tempo subsequente.
  - É assim que a memória de longo prazo é mantida.
- Assim, as **portas de entrada e de esquecimento** dão ao modelo a **capacidade de aprender quando (e o quanto) manter ou não** esse valor inalterado em resposta a entradas subsequentes.

Dinâmica das portas em uma célula LSTM



- Sempre que o_t for próximo de **1**, permitimos que o **estado interno atual da célula, c_t, impacte os instantes de tempo subsequentes** via h_t.
- Por outro lado, se o_t for próximo de **0**, impedimos que o **estado interno atual da célula, c_t, impacte os instantes de tempo subsequentes.**
- Assim
  - $o_t \rightarrow 1$: a **informação de c_t é importante** para o futuro.
  - $o_t \rightarrow 0$: a **informação de c_t não é relevante** para os próximos estados.

Porta de esquecimento

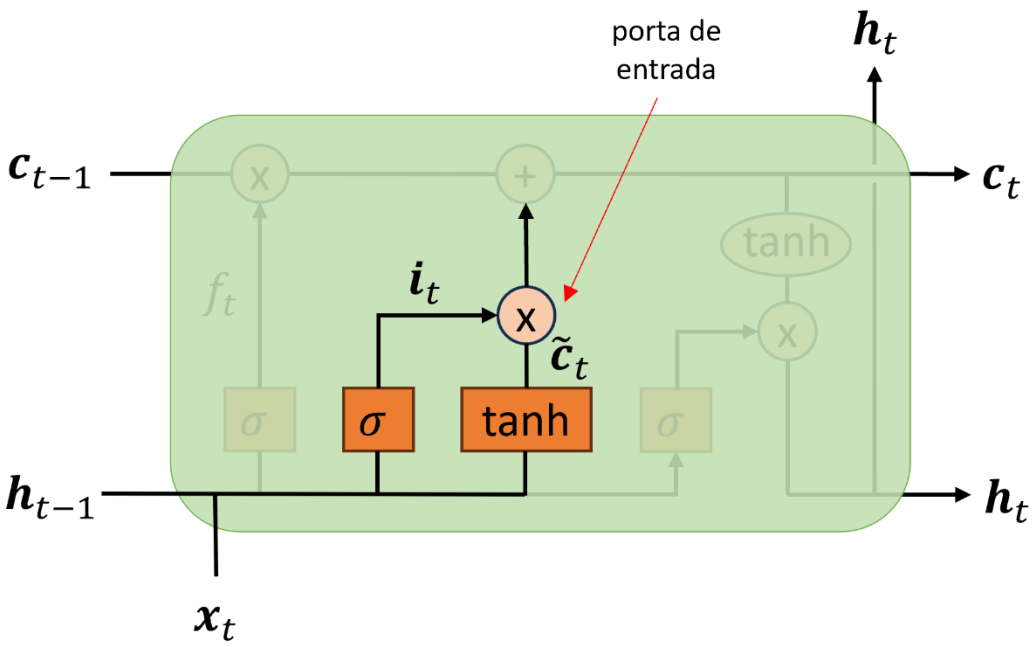


- **Controla** quais **informações** devem ser **descartadas** do estado de longo prazo

$$f_t = \begin{cases} \mathbf{0}, & \text{esquece completamente } c_{t-1} \\ \mathbf{1}, & \text{mantém 100\% de } c_{t-1} \end{cases}$$

- Porta controlada pela entrada atual x_t e pela saída anterior da célula, h_{t-1}.
- Exemplo aplicado ao PLN:
  - O **estado de longo prazo** **mantém** o **gênero do sujeito** de uma frase para usar os pronomes corretos.
  - Ao ver um **novo sujeito**, a célula **esquece** o gênero do sujeito anterior.

Porta de entrada

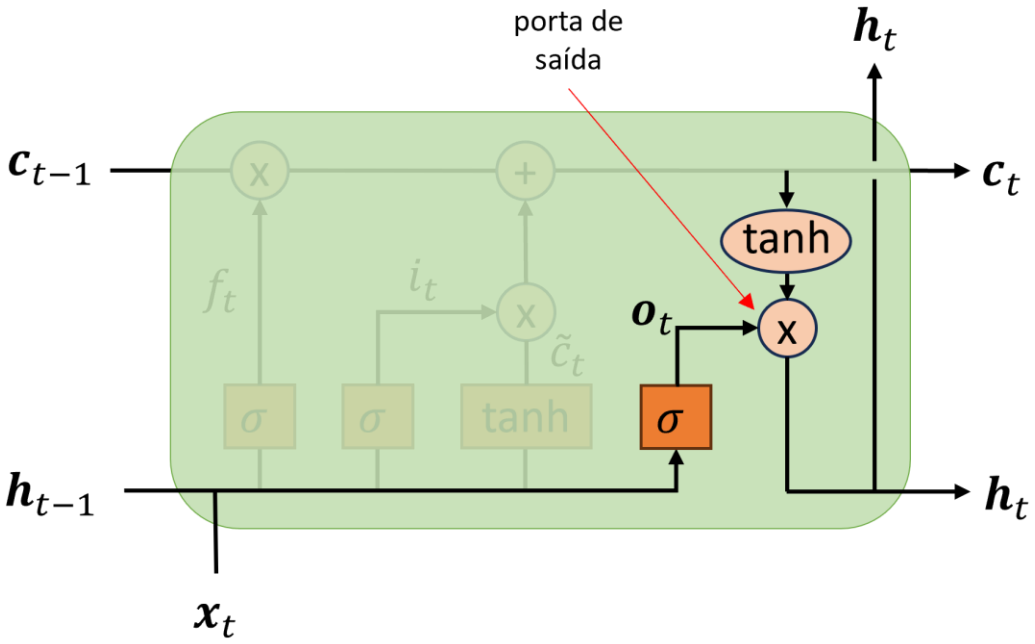


- **Controla o quanto atualizar o estado antigo**, c_{t-1}, com um novo candidato, $\tilde{c}_t$:

$$i_t = \begin{cases} 0, & \text{sem atualização} \\ 1, & \text{atualiza } c_{t-1} \text{ com } 100\% \text{ de } \tilde{c}_t \end{cases}$$

- Também controlada por x_t e h_{t-1}.
- No exemplo de PNL:
  - **Adiciona** o **gênero do novo sujeito** da frase ao **estado de longo prazo** da célula para **substituir** o antigo **que foi esquecido**.

Porta de saída



- A **saída da célula** é uma **versão filtrada** do **estado da célula**, $\boldsymbol{c_t}$, controlada por $\boldsymbol{x_t}$ e $\boldsymbol{h_{t-1}}$.
- $\boldsymbol{c_t}$ é passado através de uma função *tanh*, que comprime seus valores entre -1 e 1.
- Em seguida, a **porta decide o quanto de $\boldsymbol{c_t}$ será exposto na saída**

$$\boldsymbol{o_t} = \begin{cases} \mathbf{0}, & \text{sem saída} \\ \mathbf{1}, & \text{passa } 100\% \text{ do estado} \end{cases}$$

- No exemplo de PLN:
  - A célula **armazena** que o **sujeito é plural**. Quando um **verbo aparece**, a porta de saída **libera** essa **informação** para **escolher a conjugação correta e influenciar passos subsequentes**.

Resumo das equações de atualização da célula

- Vetor da porta de esquecimento

$$\boldsymbol{f}_t = \sigma\big([\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]\boldsymbol{W}_f + \boldsymbol{b}_f\big),$$

- Vetor da porta de entrada

$$\boldsymbol{i}_t = \sigma([\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]\boldsymbol{W}_i + \boldsymbol{b}_i),$$

- Vetor da porta de saída

$$\boldsymbol{o}_t = \sigma([\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]\boldsymbol{W}_o + \boldsymbol{b}_o),$$

- Vetor de informação candidata

$$\tilde{\boldsymbol{c}}_t = \tanh([\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]\boldsymbol{W}_c + \boldsymbol{b}_c)\,,$$

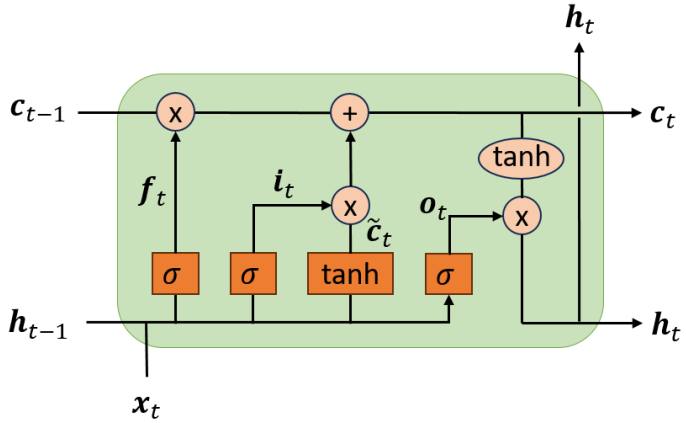
- Vetor de estado de longo prazo

$$\boldsymbol{c}_t = \underbrace{\boldsymbol{f}_t \boldsymbol{c}_{t-1}}_{\text{controla o esquecimento da informação anterior}} + \underbrace{\boldsymbol{i}_t \tilde{\boldsymbol{c}}_t}_{\text{controla a entrada de nova informação}},$$

- Vetor de estado de curto prazo

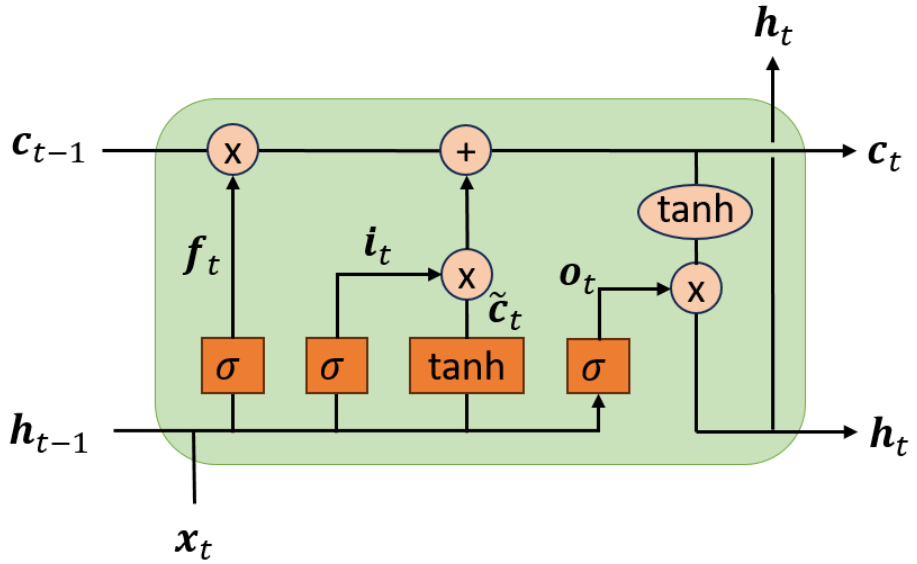
$$\boldsymbol{h}_t = \tanh(\boldsymbol{c}_t) \odot \boldsymbol{o}_t,$$

onde $\odot$ é o operador de multiplicação elemento a elemento.



Long-short term memory (LSTM)

- Resumindo, uma célula LSTM pode:
  - aprender a **reconhecer uma entrada importante e armazená-la** (totalmente ou parcialmente) no estado de longo prazo, $\boldsymbol{C}_t$,
    - Essas são as funções da **porta de entrada**
  - **preservá-la** pelo tempo que for necessário
    - Essa é a função da **porta de esquecimento**
  - e **expô-la** sempre que necessário
    - Essa é a função da **porta de saída**

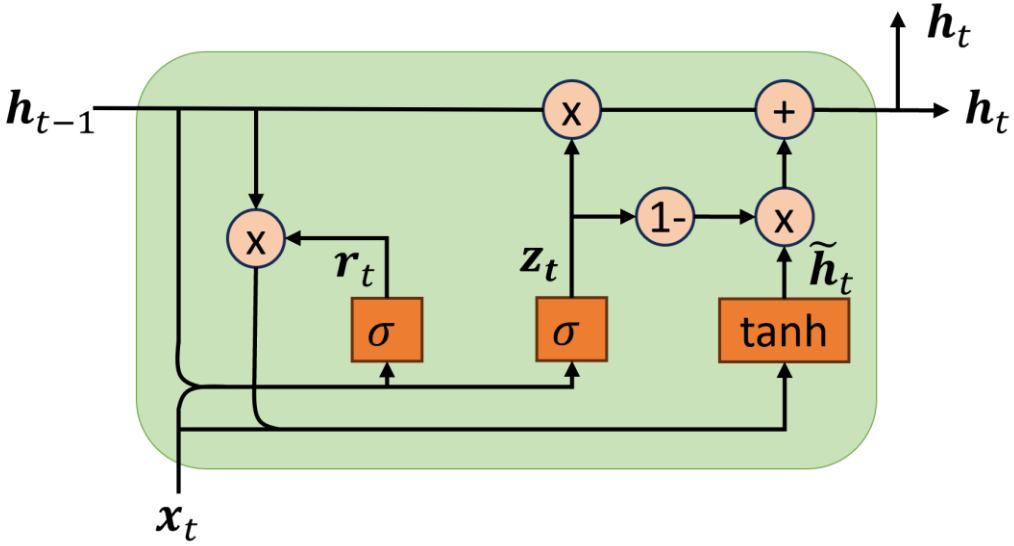


Principais desvantagens de LSTMs

- As saídas das portas **nunca são realmente 0 ou 1**.
  - Assim, o **conteúdo da célula inevitavelmente se corrompe após um longo período**.
  - Portanto, ainda apresenta **dificuldades com dependências temporais muito longas**.
- Alta **complexidade computacional**.
  - **Muito mais parâmetros** do que as RNNs tradicionais.
- **Dificuldade de paralelização**
  - **Processamento sequencial**: a célula depende de $t-1$ para calcular t.
  - Difícil explorar paralelismo em GPU.
- Hoje é **superada em muitos cenários**
  - Modelos mais modernos, como *Transformers* com *attention*, apresentam **melhor paralelização** e **capturam de dependências globais**.

Será que conseguimos algo quase tão bom quanto uma LSTM, porém mais simples?

Gated recurrent unit (GRU)

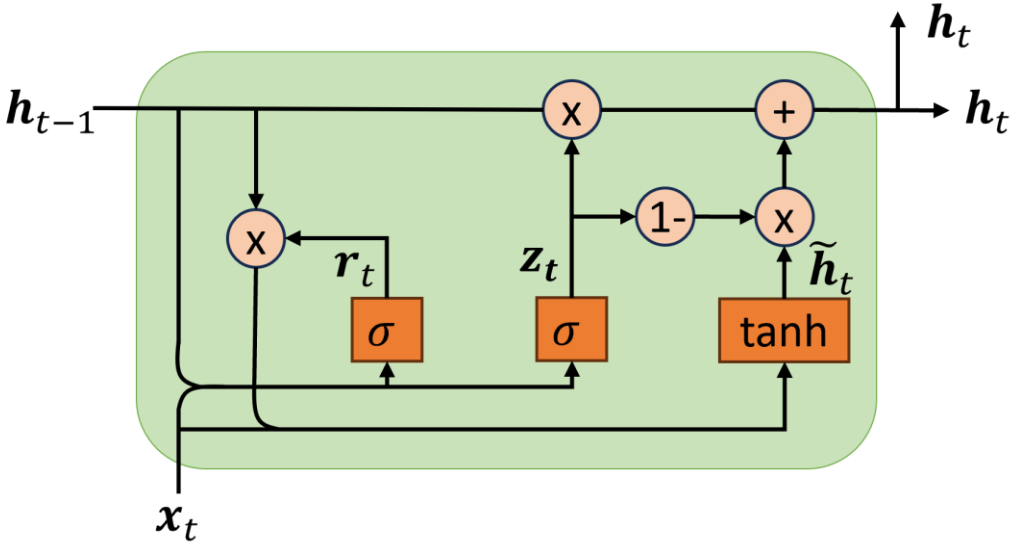


- Com o sucesso das LSTMs em capturar dependências de longo prazo, novas arquiteturas foram propostas com o objetivo de
  - reduzir a quantidade de parâmetros,
  - reduzir o custo computacional,
  - reduzir o consumo de memória,
  - e acelerar o treinamento

em relação às LSTMs.

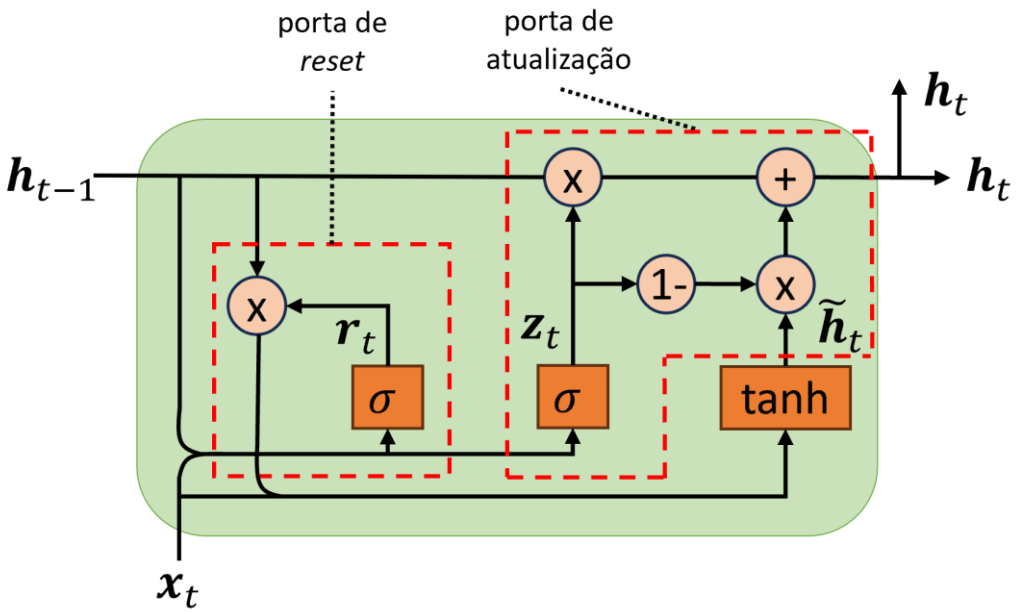
- Assim, as GRUs foram **propostas em 2014** por Kyunghyun Cho e colegas com esse objetivo.

Gated recurrent unit (GRU)



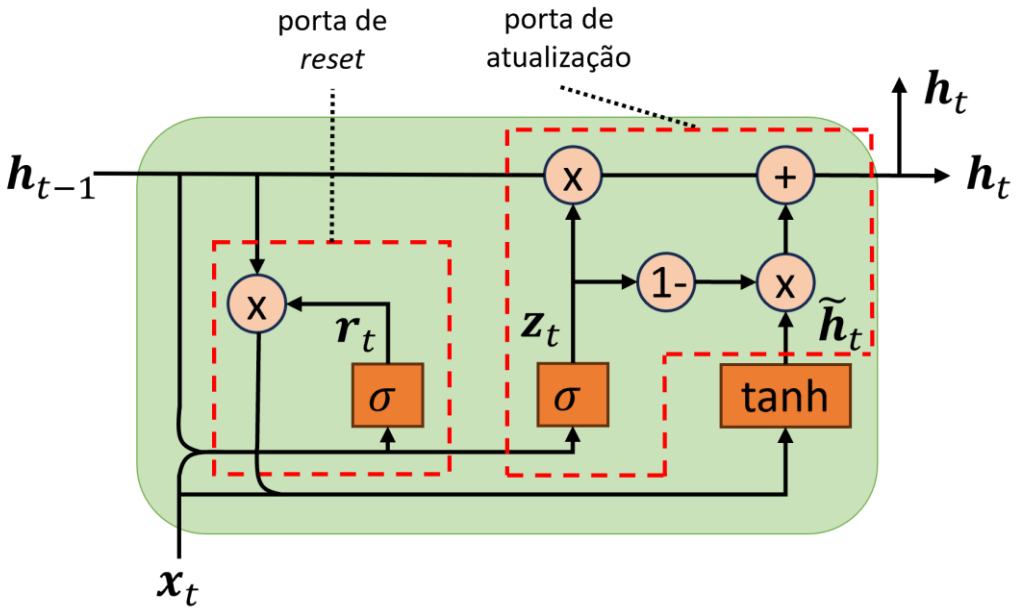
- As GRUs são uma **versão simplificada** da célula de memória LSTM.
- Em geral, **atingem desempenho comparável** às LSTMs, mas com a vantagem de terem **menos parâmetros**, serem **menos complexas** e serem **treinadas mais rapidamente**.
- Elas têm **apenas duas portas**: a de *reset* e a de atualização (controladas por $\boldsymbol{x}_t$ e $\boldsymbol{h}_{t-1}$).
- Além disso, elas **incorporam a memória de longo prazo no próprio estado oculto**.

Gated recurrent unit (GRU)



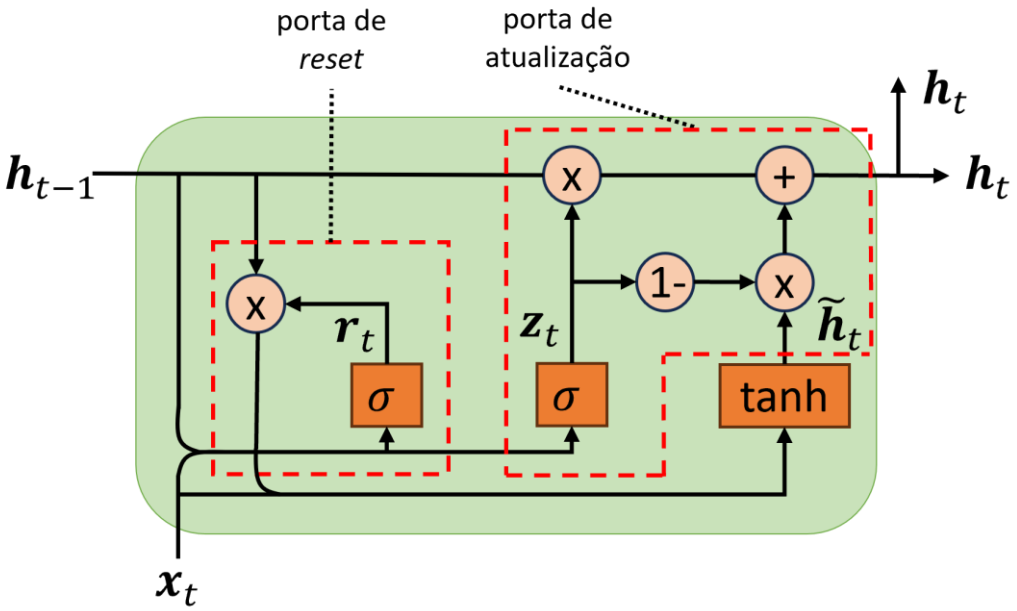
- A **porta de atualização** é responsável pela **memória de longo prazo**.
- Ela é a **combinação** das **portas de esquecimento** e de **entrada** da LSTM.
- Um **único vetor**, z_t, **controla** tanto a **porta de esquecimento** quanto a **porta de entrada**.

Gated recurrent unit (GRU)



- Se $z_t = 1$, a **porta de esquecimento** é **aberta** e a **porta de entrada** é **fechada** ($1 - z_t = 0$).
  - Retem-se o estado anterior, h_{t-1}.
- Se $z_t = 0$, a **porta de esquecimento** é **fechada** e a **porta de entrada** é **aberta** ($1 - z_t = 1$).
  - O estado oculto passa a ser $\tilde{h}_t$.
- Em outras palavras, sempre que uma **informação precisa ser armazenada**, o **local** (i.e., elemento) **onde ela será armazenada é apagado primeiro**.

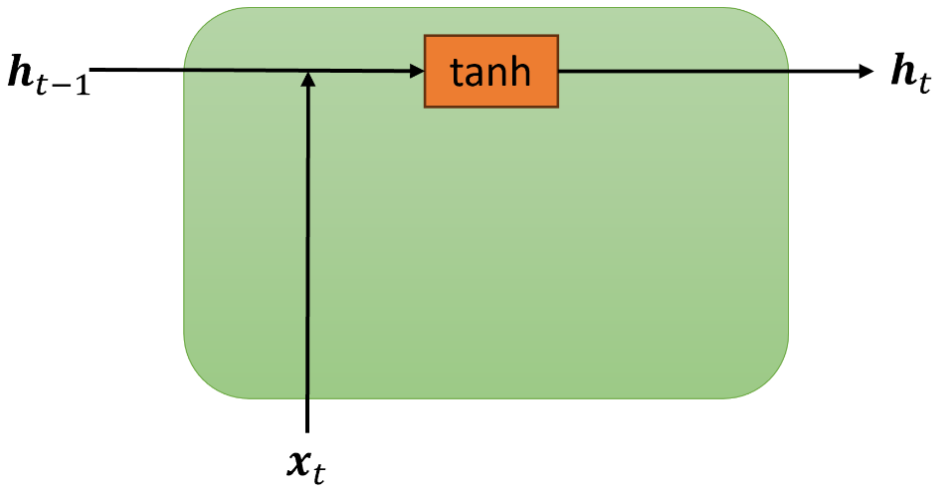
Gated recurrent unit (GRU)



- A porta de *reset* é responsável pela **memória de curto prazo**.
- Ela decide **quanta informação passada**, $\boldsymbol{h_{t-1}}$, será **mantida ou descartada**.
- A célula GRU **não tem uma porta de saída**.
- **O vetor de estado oculto**, $\boldsymbol{h_t}$, **completo é apresentado na saída** da célula a cada passo de tempo.

Como as células LSTM e GRU minimizam o problema do desaparecimento do gradiente e, consequentemente, mantém a memória de longo prazo?

Desaparecimento do gradiente

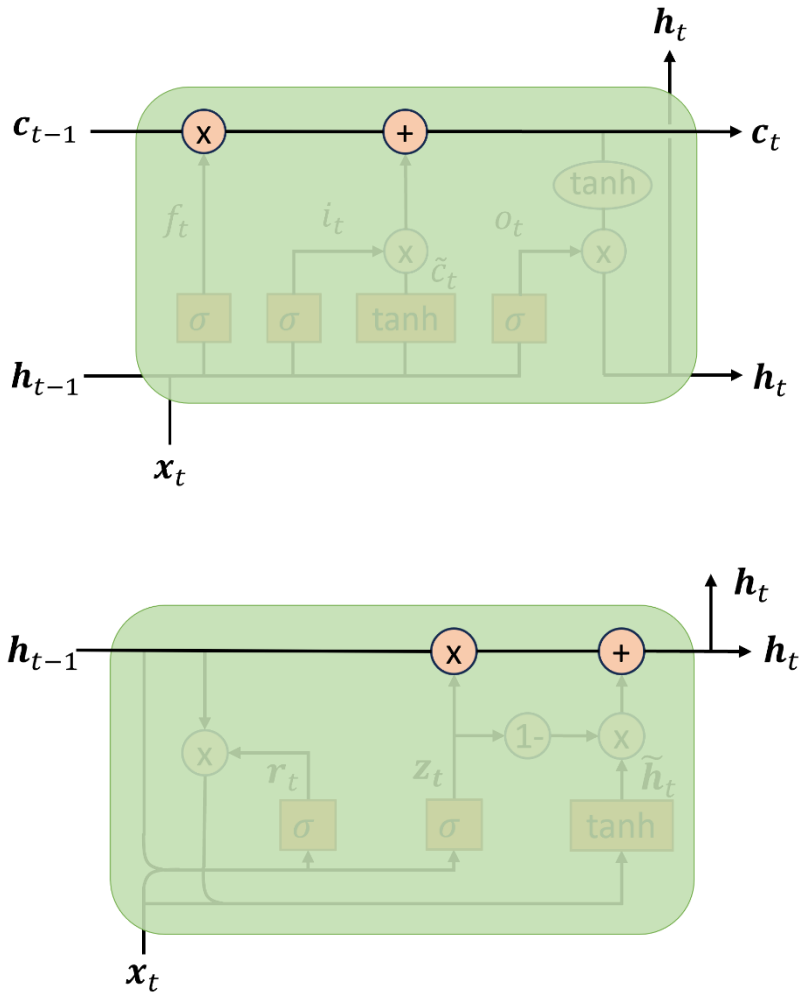


- As células de RNNs tradicionais apresentam um **caminho não-linear para a retropropagação do gradiente**.
- O **novo estado da célula passa por uma função tanh**

$$\boldsymbol{h}_t = \tanh(\boldsymbol{W}[\boldsymbol{x}_t, \boldsymbol{h}_{t-1}]) \,.$$

- Como sabemos, a derivada da função tanh tem um valor máximo igual 1.
- Como vimos, o resultado é que o **gradiente diminui exponencialmente** à medida que volta no tempo, **impedindo que a rede aprenda dependências de longo prazo**.

Desaparecimento do gradiente



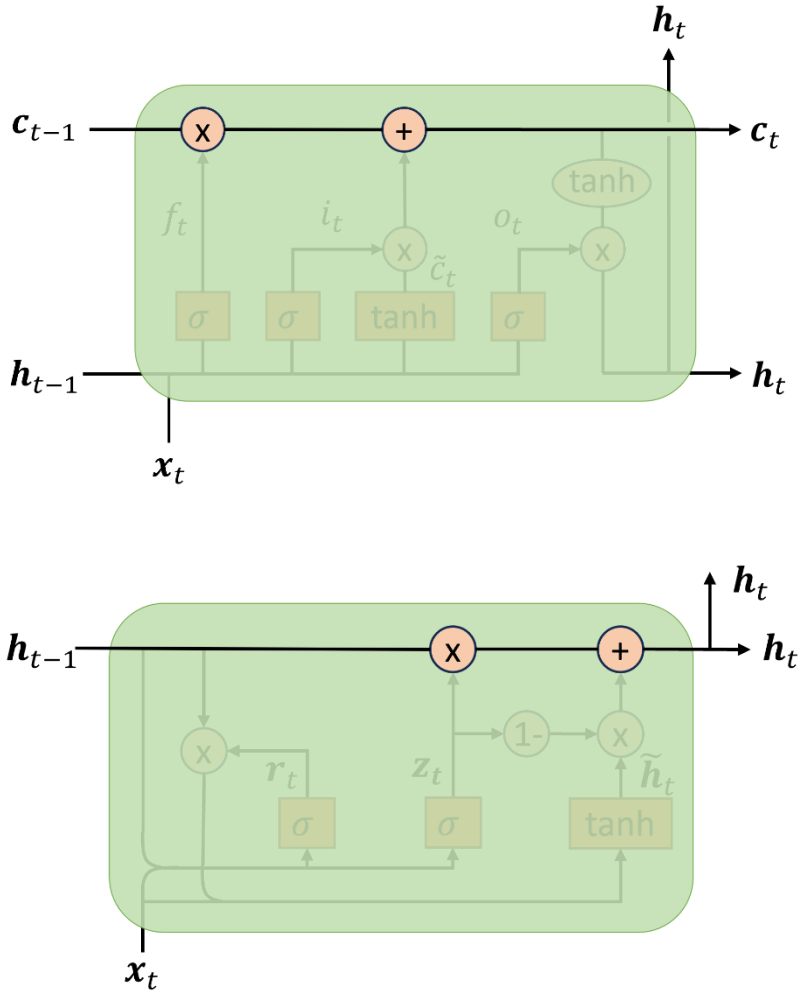
- Já as células de LSTMs e GRUs **modificam o estado da célula, $\boldsymbol{c}_t$, por meio de um caminho controlável** (por $\boldsymbol{f}_t$, $\boldsymbol{i}_t$ e $\boldsymbol{z}_t$) que **contém apenas interações lineares**

$$\boldsymbol{c}_t = \boldsymbol{f}_t \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \tilde{\boldsymbol{c}}_t \text{ (LSTM)}$$

$$\boldsymbol{h}_t = \boldsymbol{z}_t \boldsymbol{h}_{t-1} + (\mathbf{1} - \boldsymbol{z}_t) \tilde{\boldsymbol{h}}_t \text{ (GRU)}$$

- $\boldsymbol{c}_t$ e $\boldsymbol{h}_t$ são atualizados usando-se **uma soma e multiplicações elemento a elemento**, **sem passar por transformações não lineares**.

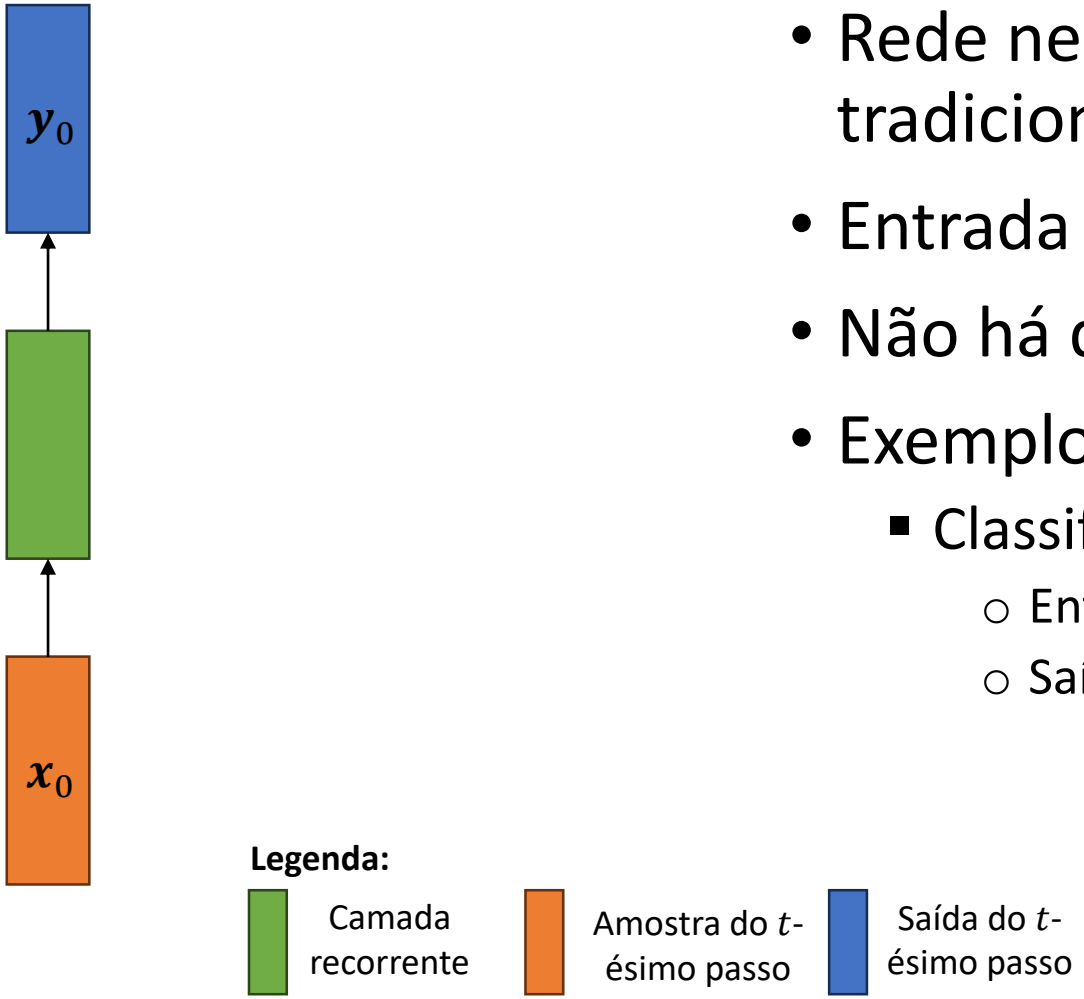
Desaparecimento do gradiente



- Esse **caminho controlável** **permite que os gradientes retrocedam por longos instantes de tempo** sem **desaparecer**.
- A rede **aprende** a decidir quando deixar o **gradiente desaparecer** e quando **preservá-lo**.
- Portanto, as portas permitem que a rede decida o que guardar e o que descartar.

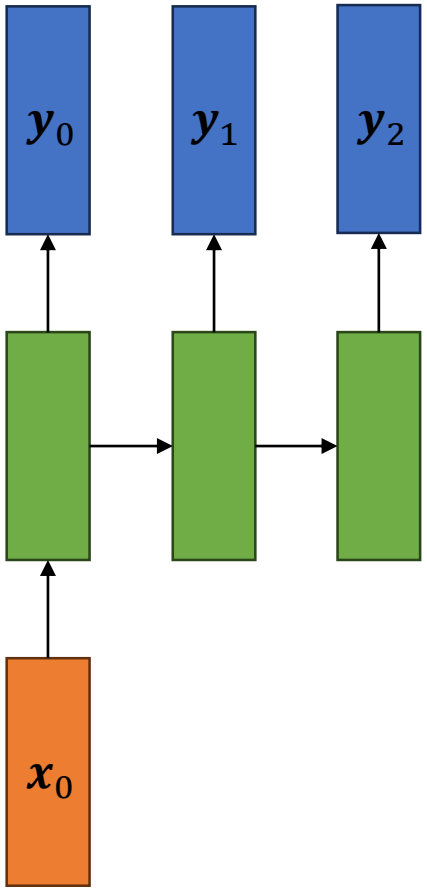
Diferentes aplicações para as RNNs

Aplicação: um-para-um



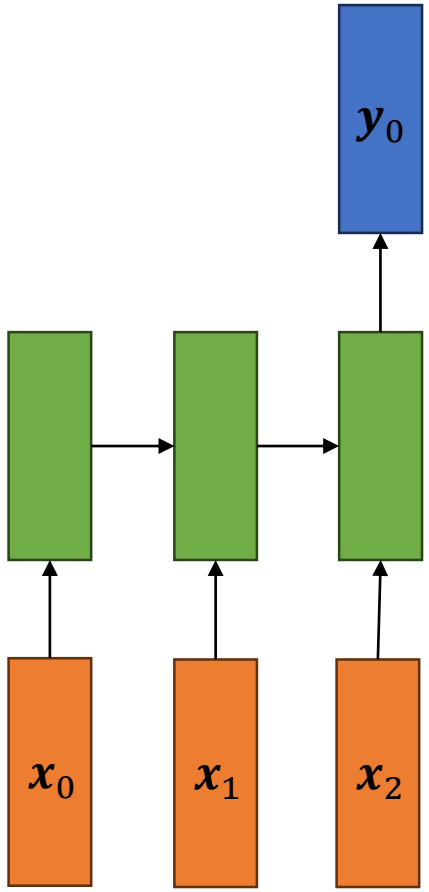
- Rede neural de alimentação direta tradicional.
- Entrada e saída de tamanhos fixos.
- Não há dados sequenciais envolvidos.
- Exemplo de uso:
  - Classificação de imagens.
    - Entrada: imagem
    - Saída: classe

Aplicação: um-para-vários



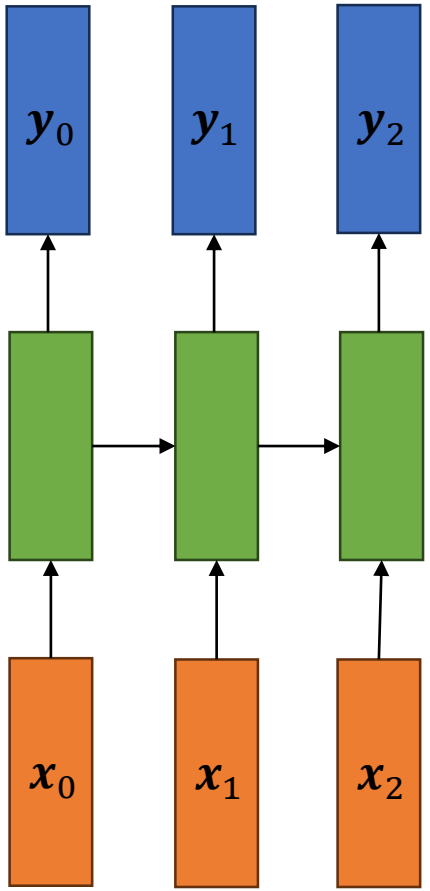
- Entrada de tamanho fixo.
- Saída de tamanho variável (sequência).
- Rede do tipo vetor para sequência
- Exemplo de uso:
  - Legendas de imagens.
    - Entrada: uma imagem
    - Saída: sequência de palavras descrevendo a imagem
  - Geração de música.
    - Entrada: um vetor que representa um gênero musical
    - Saída: as amostras de áudio da música

Aplicação: vários-para-um



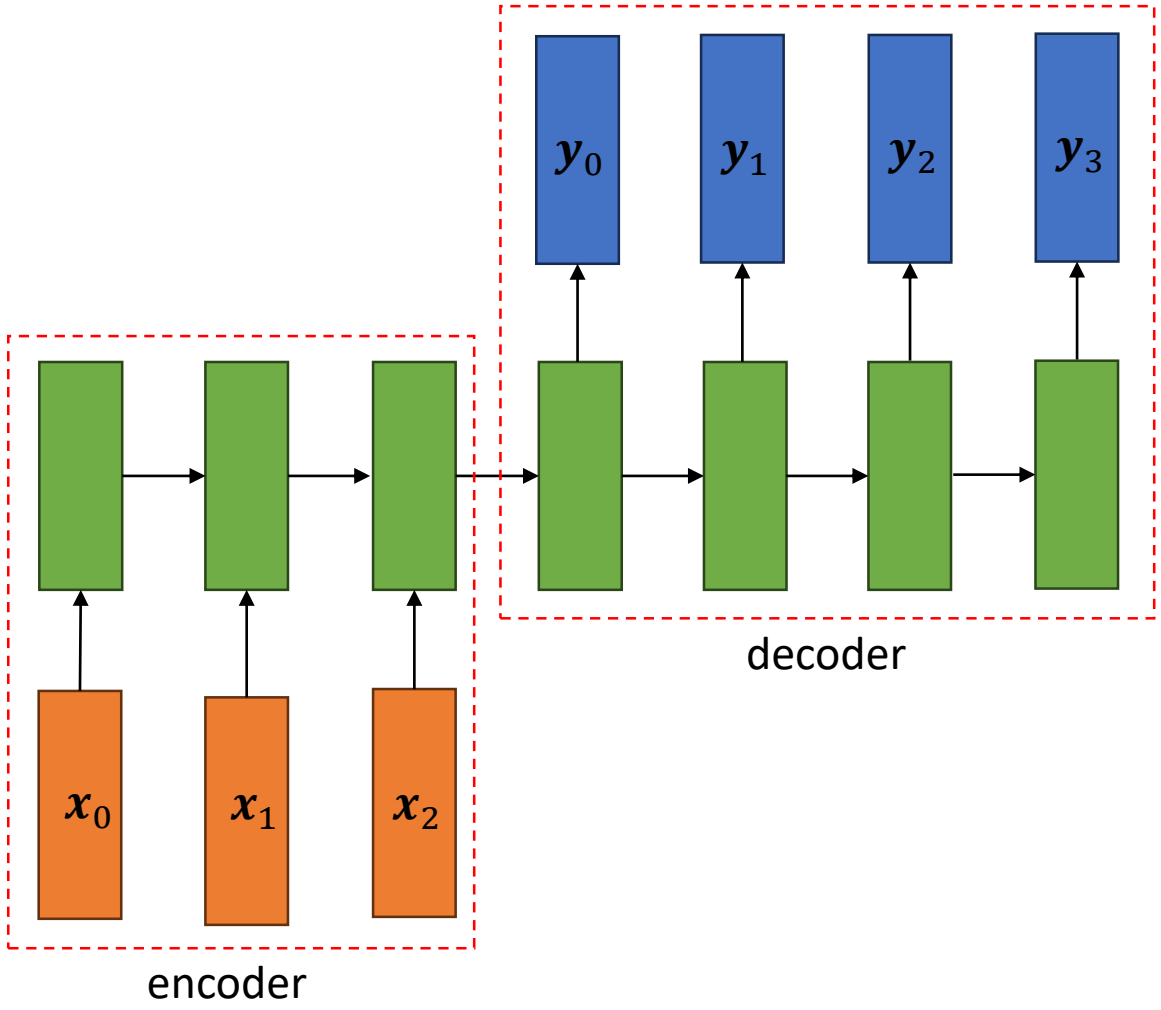
- Entrada de tamanho variável (sequência).
- Saída de tamanho fixo.
- Rede do tipo sequência para vetor.
- Exemplo de uso:
  - Classificação de sentimentos: por exemplo, classificar o sentimento de uma crítica de filme:
    - Entrada: sequência de palavras correspondentes à crítica
    - Saída: pontuação da crítica: 0 [odiou] até 1 [amou]

Aplicação: vários-para-vários síncrono



- Entrada de tamanho variável (sequência).
- Saída de tamanho variável (sequência).
- Rede do tipo sequência para sequência
- Entrada e saída sincronizadas.
  - Comprimentos de entrada e saída são iguais.
- Exemplo de uso:
  - Classificação de vídeo quadro a quadro.
    - Monitoramento de tráfego:
      - ✓ Entrada: sequência de imagens da vista de um semáforo.
      - ✓ Saída: quantidade de carros em cada imagem.

Aplicação: vários-para-vários assíncrono



- Entrada de tamanho variável (sequência).
- Saída de tamanho variável (sequência).
- Rede do tipo *encoder/decoder*.
- Entrada e saída não são sincronizadas.
  - Comprimentos de entrada e saída não são necessariamente iguais.
- Exemplo de uso:
  - Tradução automática
    - Entrada: sequência de palavras em uma língua.
    - Saída: sequência de palavras em outra língua.
  - *chatbots*

Exemplo

- Exemplo: rnn.ipynb



Lista de exercícios

Lista #11 - RNN

Perguntas?

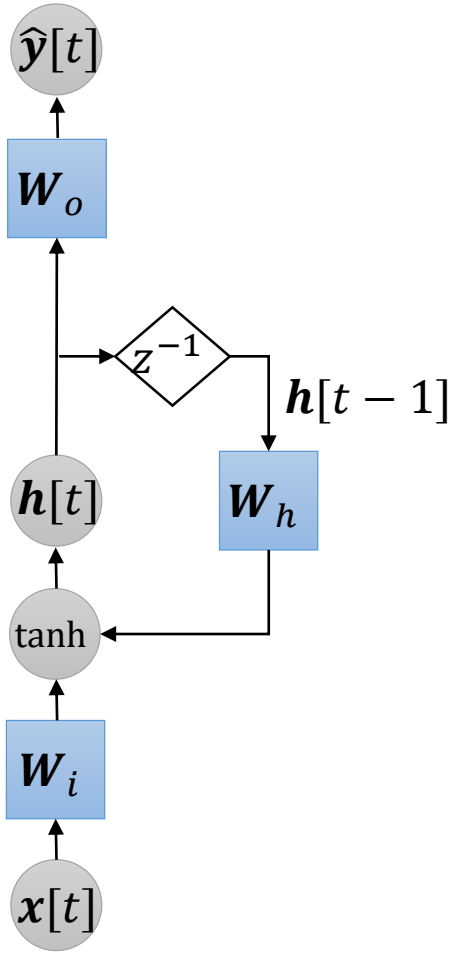
Obrigado!

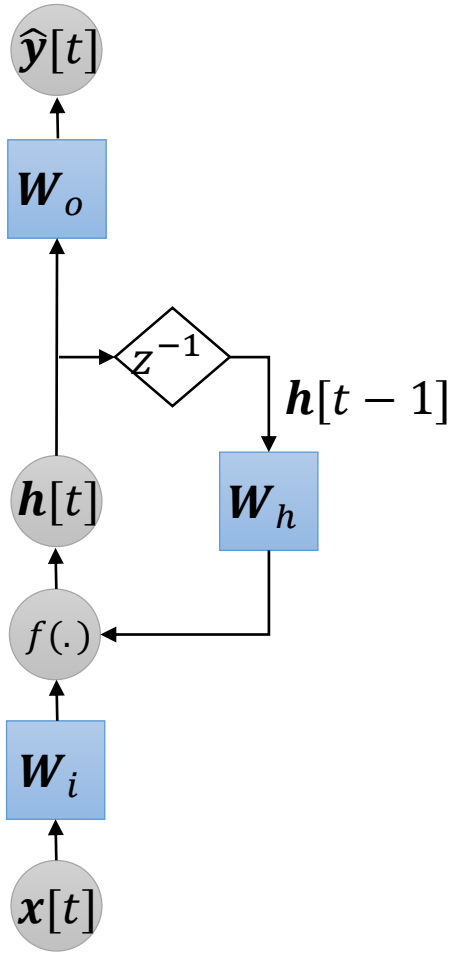
Por que não usar funções de ativação ReLU?

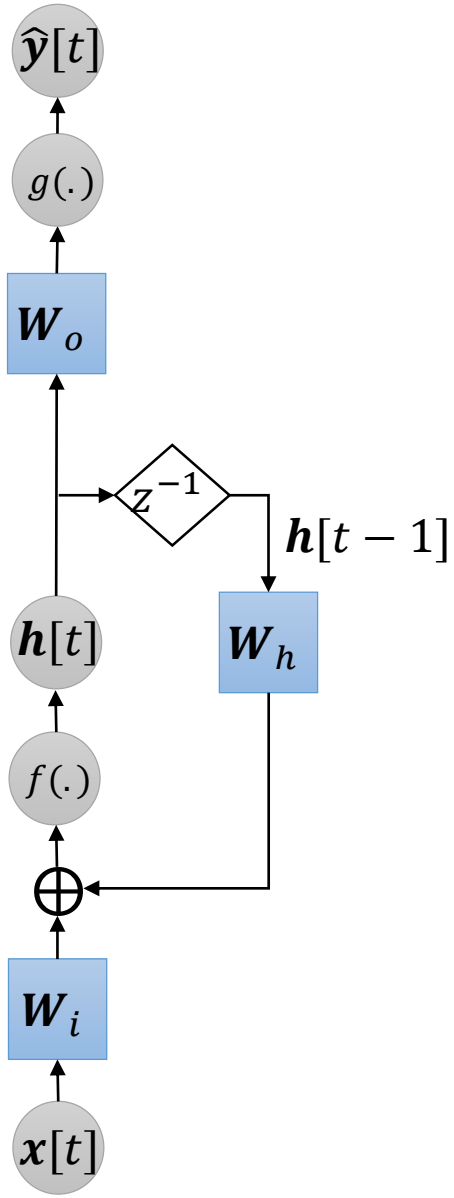
Por que não usar funções de ativação ReLU?

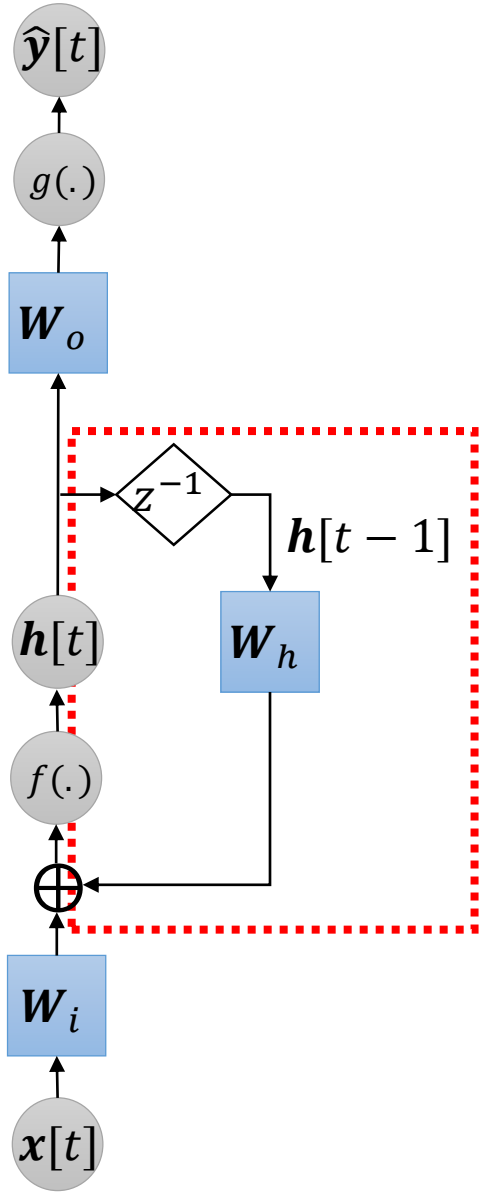
- RNNs **reusam** a **mesma matriz de pesos e a mesma função de ativação repetidamente** em cada passo.
- **Valores maiores que 1** na entrada de uma **função que não satura** como a ReLU ($\boldsymbol{y} = \max(\boldsymbol{x}, 0)$ podem causar um crescimento exponencial (explosão) dos valores do estado oculto, $h[t]$.
- Isso torna o treinamento instável:
  - Os estados podem crescer sem limite, causando dinâmica instável no tempo.
  - Em **sistemas dinâmicos** (como em RNNs), isso é ruim, pois **causa oscilações ou divergência**.
- Além disso, **valores negativos** em sua entrada **geram saídas e derivadas iguais a 0**, fazendo com que o **fluxo de aprendizado morra**.

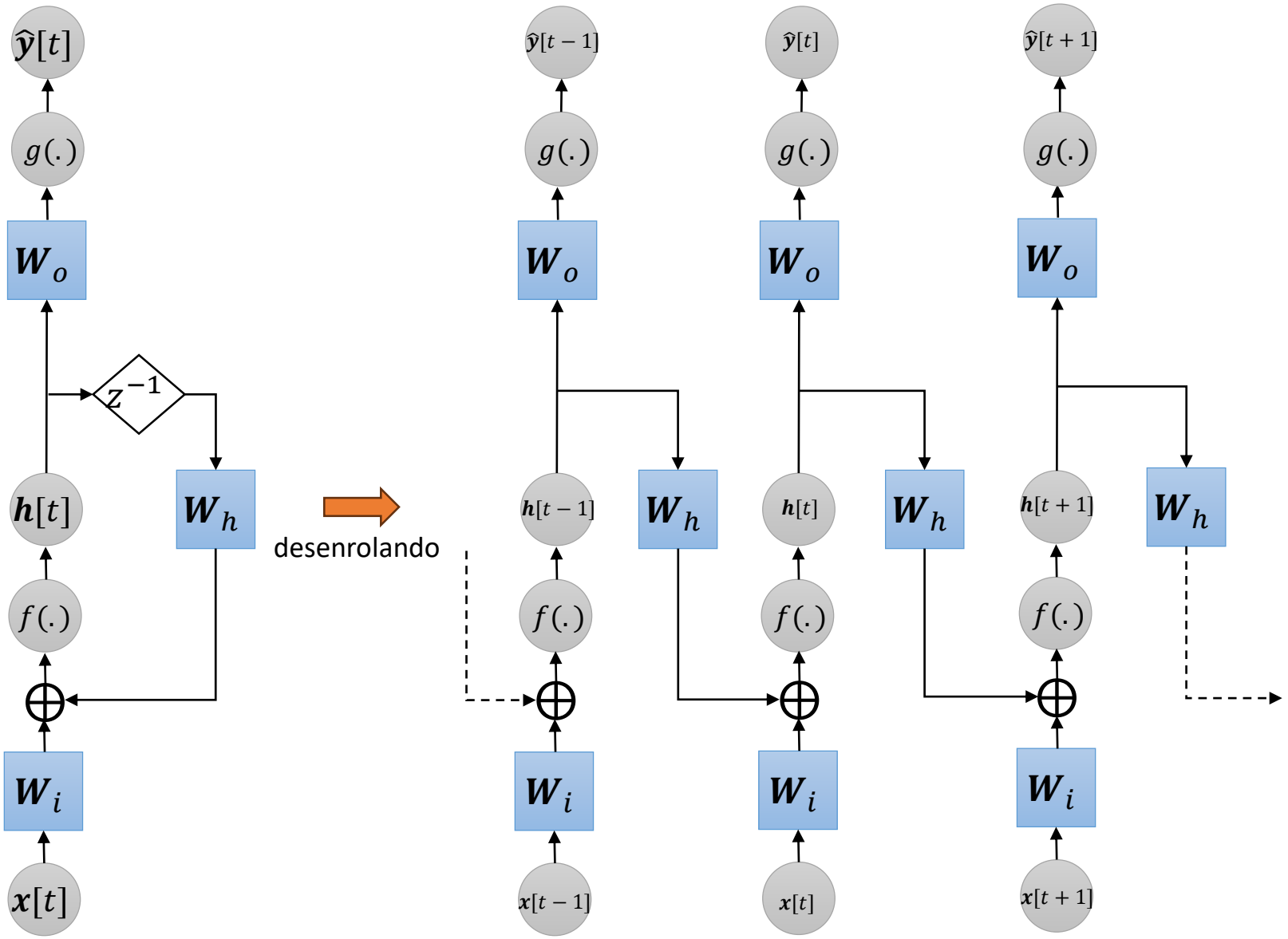
Figuras

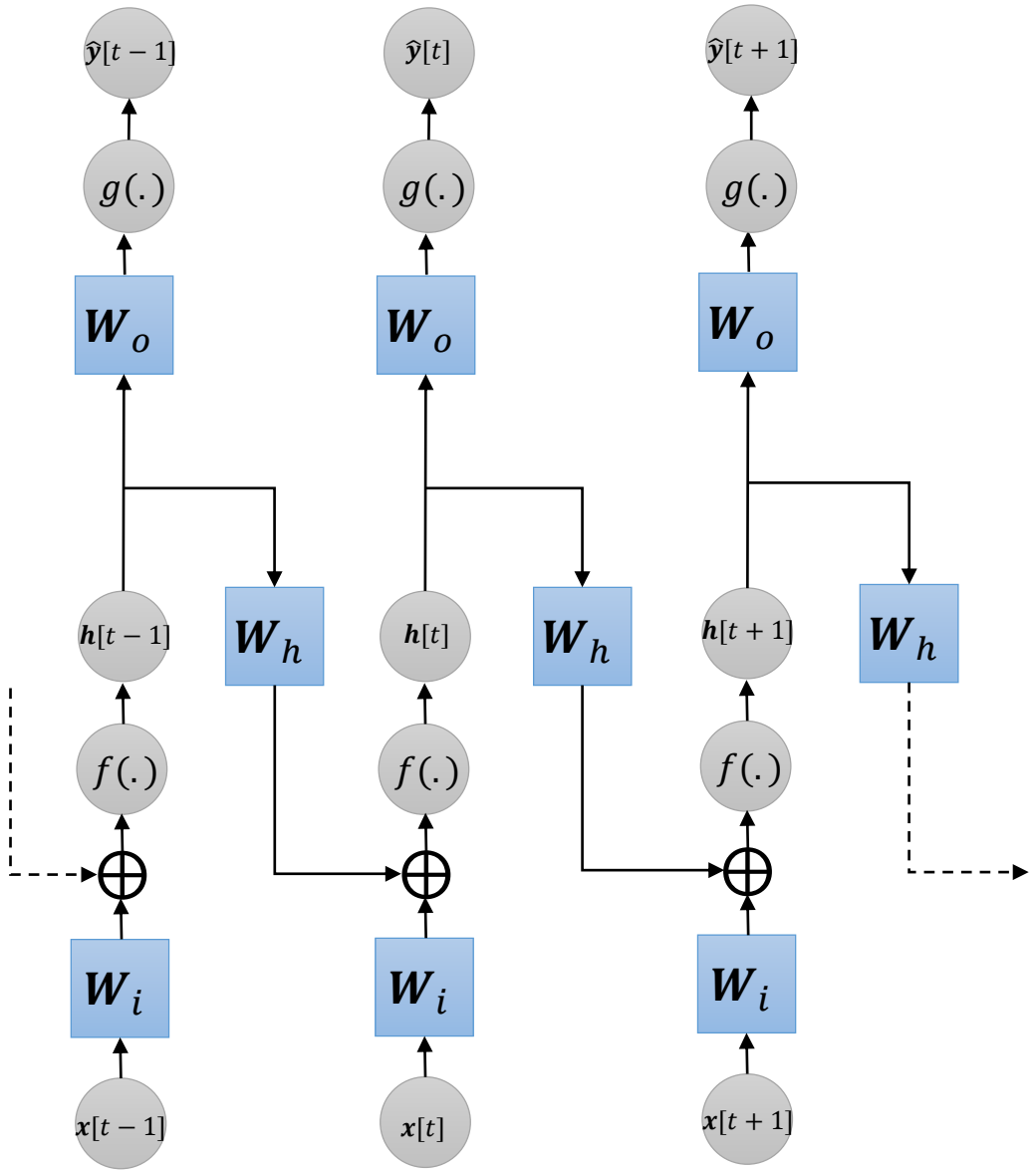

$\hat{\boldsymbol{y}}[t]$
$\boldsymbol{W}_o$
z^{-1}
$\boldsymbol{h}[t-1]$
$\boldsymbol{h}[t]$
$\boldsymbol{W}_h$
tanh
$\boldsymbol{W}_i$
$\boldsymbol{x}[t]$

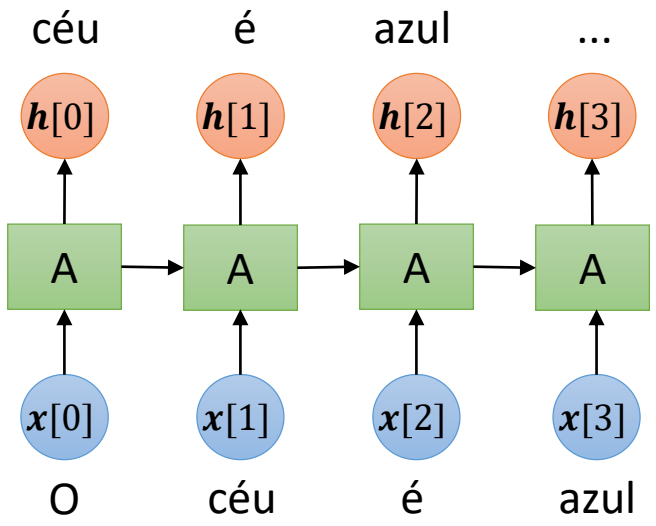

$\hat{y}[t]$
W_o
z^{-1}
$h[t-1]$
$h[t]$
W_h
$f(.)$
W_i
$x[t]$

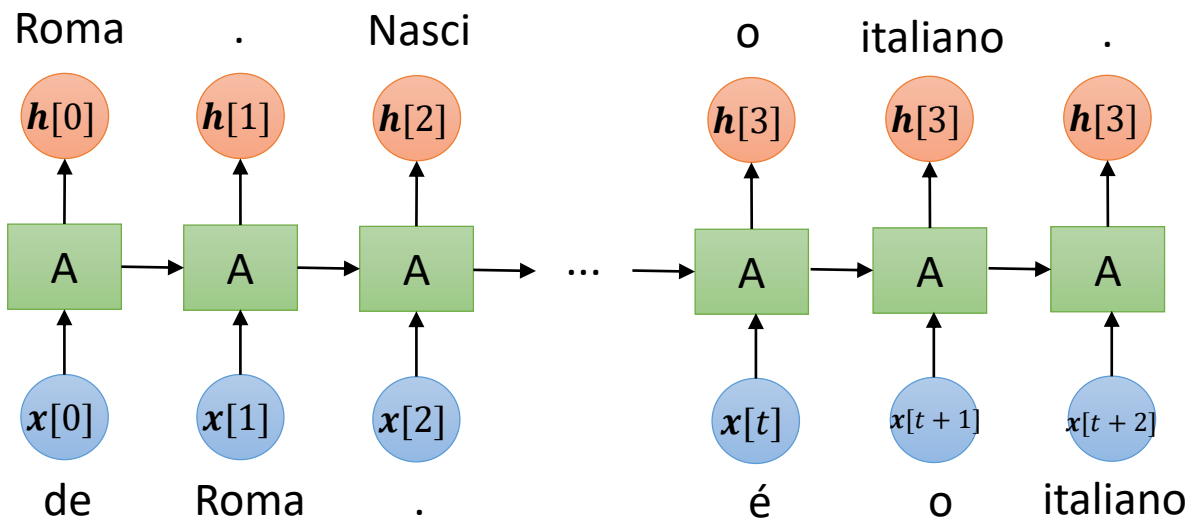

$\hat{y}[t]$
$g(.)$
W_o
z^{-1}
$h[t-1]$
$h[t]$
W_h
$f(.)$
W_i
$x[t]$

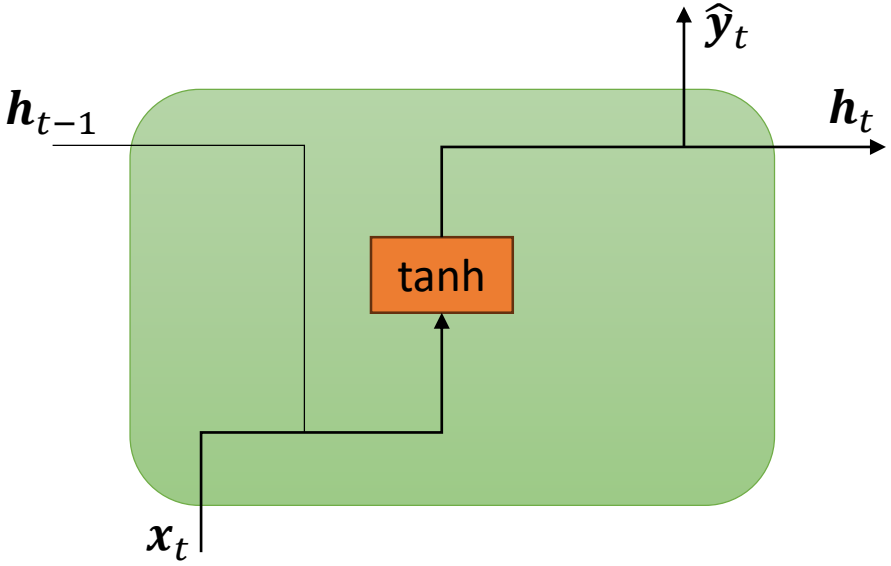

$\hat{y}[t]$
$g(.)$
W_o
z^{-1}
$h[t-1]$
$h[t]$
W_h
$f(.)$
W_i
$x[t]$

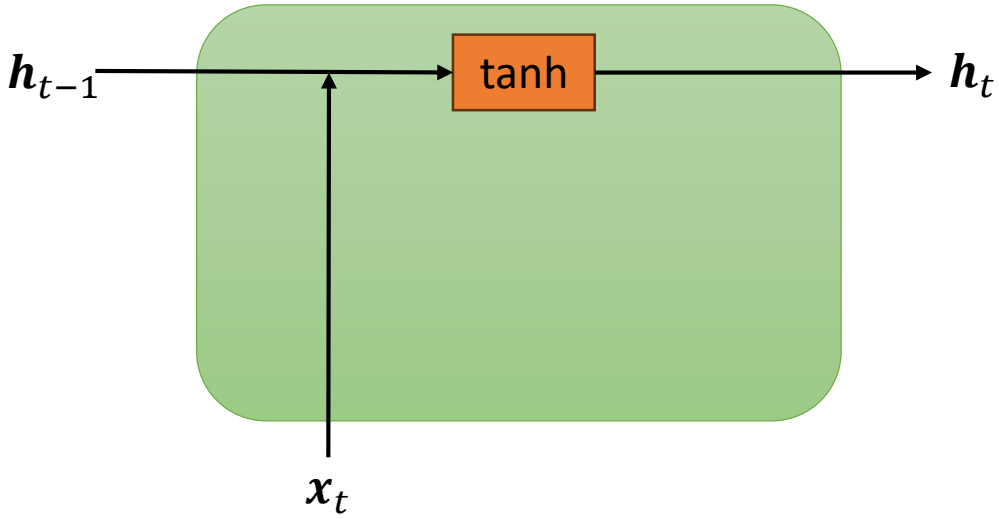

$\hat{y}[t]$
$g(.)$
W_o
z^{-1}
$h[t]$
W_h
$f(.)$
W_i
$x[t]$
desenrolando
$\hat{y}[t-1]$
$\hat{y}[t]$
$\hat{y}[t+1]$
$h[t-1]$
$h[t+1]$
$x[t-1]$
$x[t+1]$

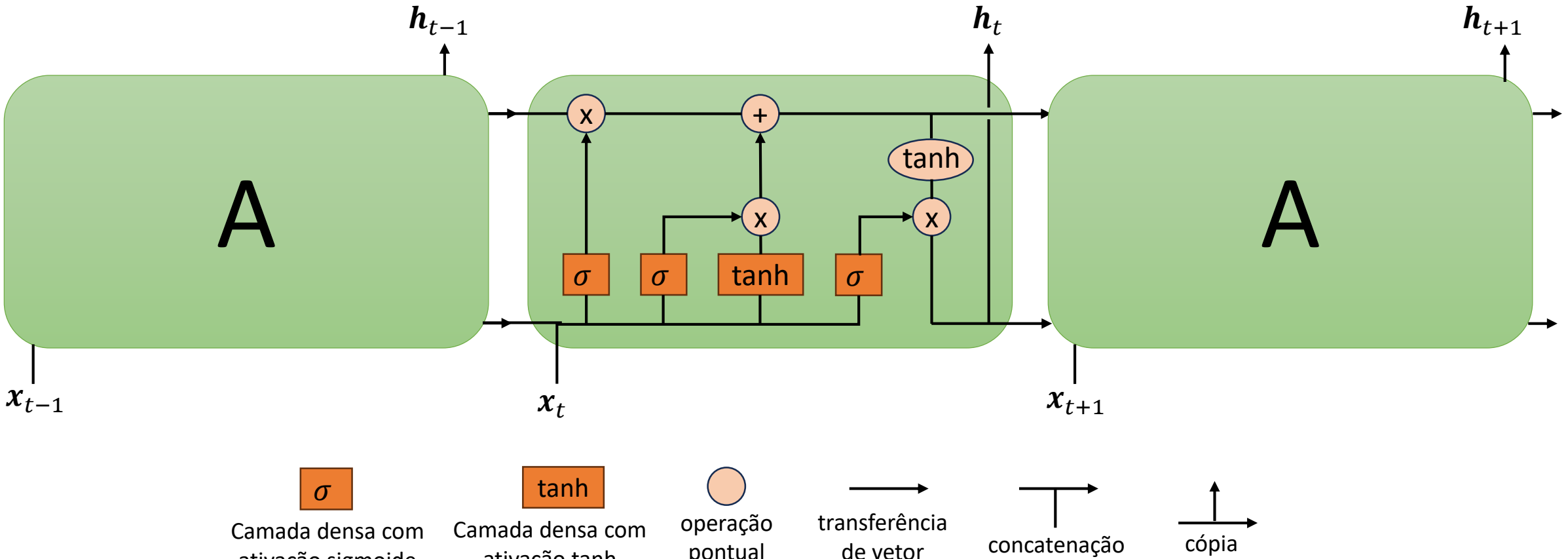

$\hat{\boldsymbol{y}}[t-1]$
$\hat{\boldsymbol{y}}[t]$
$\hat{\boldsymbol{y}}[t+1]$
$g(.)$
$g(.)$
$g(.)$
$\boldsymbol{W}_o$
$\boldsymbol{W}_o$
$\boldsymbol{W}_o$
$\boldsymbol{h}[t-1]$
$\boldsymbol{W}_h$
$\boldsymbol{h}[t]$
$\boldsymbol{W}_h$
$\boldsymbol{h}[t+1]$
$\boldsymbol{W}_h$
$f(.)$
$f(.)$
$f(.)$
$\boldsymbol{W}_i$
$\boldsymbol{W}_i$
$\boldsymbol{W}_i$
$\boldsymbol{x}[t-1]$
$\boldsymbol{x}[t]$
$\boldsymbol{x}[t+1]$

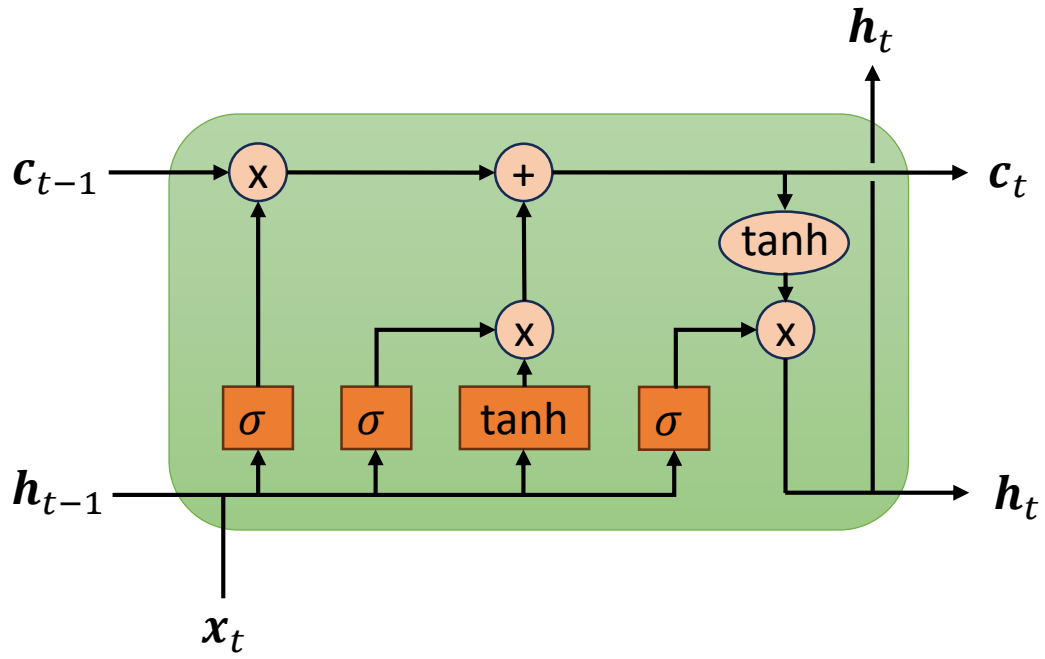

céu
é
azul
...
$\boldsymbol{h}[0]$
$\boldsymbol{h}[1]$
$\boldsymbol{h}[2]$
$\boldsymbol{h}[3]$
A
A
A
A
$\boldsymbol{x}[0]$
$\boldsymbol{x}[1]$
$\boldsymbol{x}[2]$
$\boldsymbol{x}[3]$
O
céu
é
azul


Roma
.
Nasci
o
italiano
.
h[0]
h[1]
h[2]
h[3]
h[3]
h[3]
A
A
A
...
A
A
A
x[0]
x[1]
x[2]
x[t]
x[t + 1]
x[t + 2]
de
Roma
.
é
o
italiano


$\hat{y}_t$
h_{t-1}
h_t
tanh
x_t


h_{t-1}
tanh
h_t
x_t


h_{t-1}
h_t
h_{t+1}
A
A
x
+
tanh
x
x
σ
σ
tanh
σ
x_{t-1}
x_t
x_{t+1}
σ
Camada densa com ativação sigmoide
tanh
Camada densa com ativação tanh
operação pontual
transferência de vetor
concatenação
cópia



σ
Camada totalmente conectada com ativação sigmoide

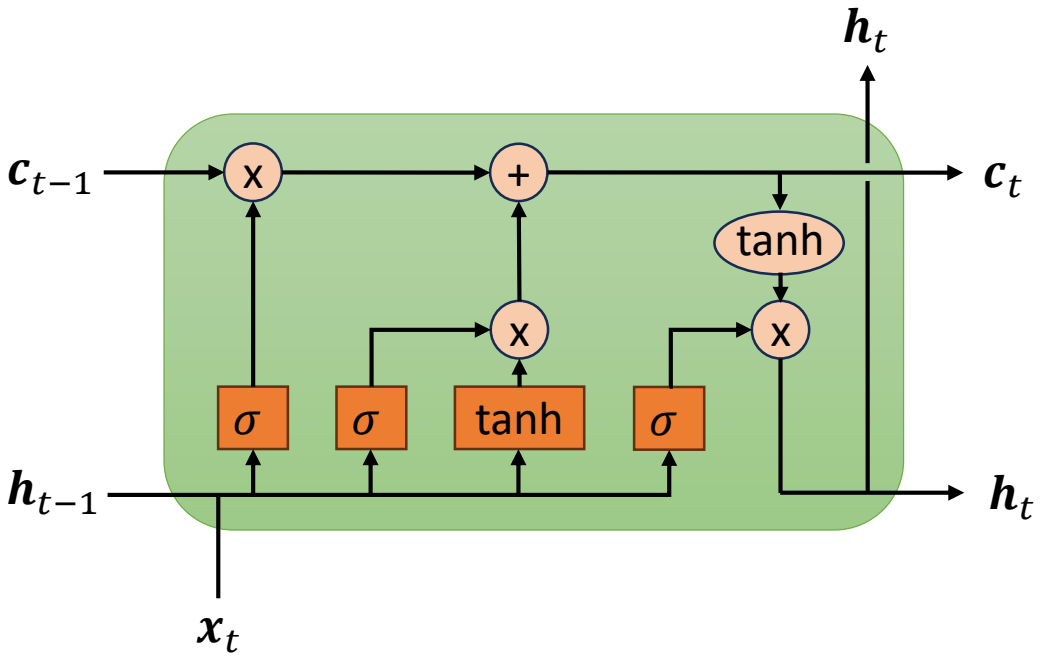
tanh
Camada totalmente conectada com ativação tanh

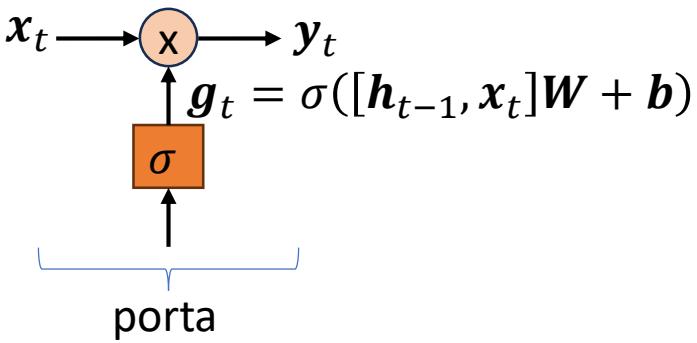
operação elemento a elemento

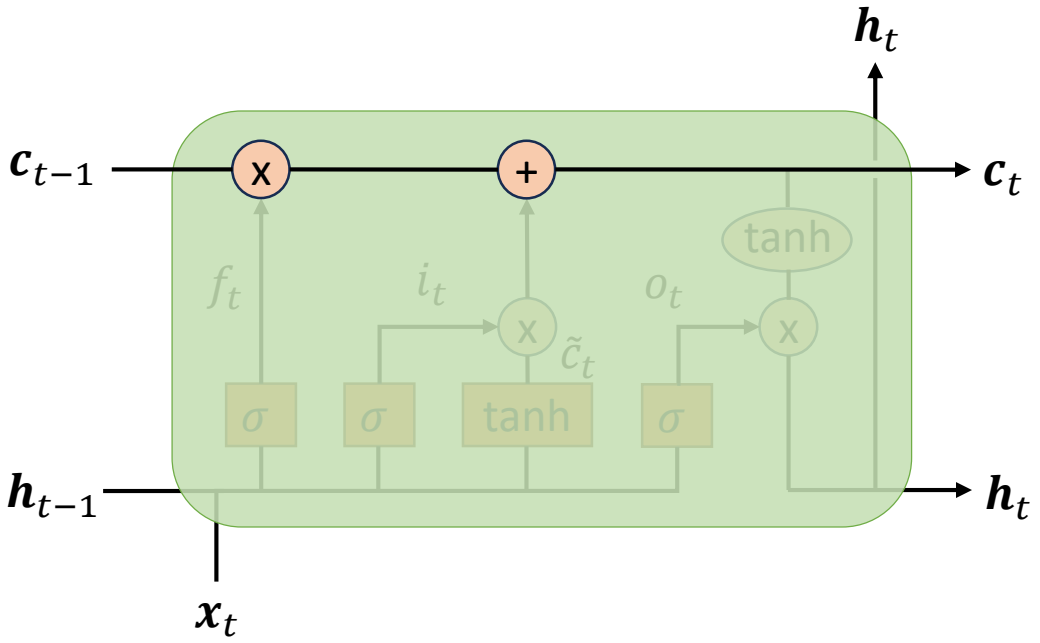
transferência de vetor

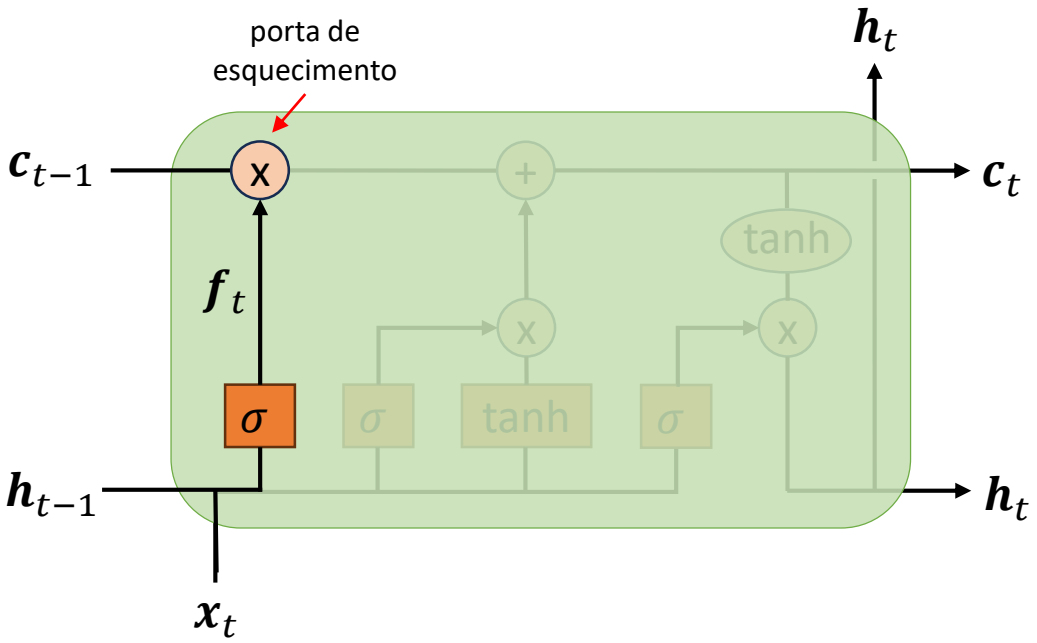
concatenação de vetores

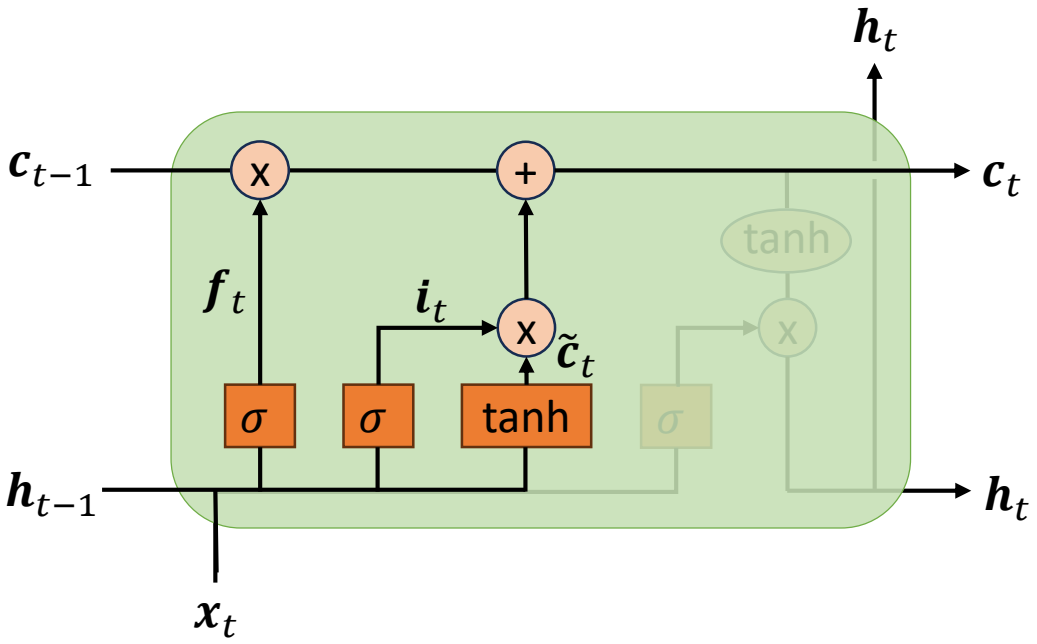
cópia de vetor

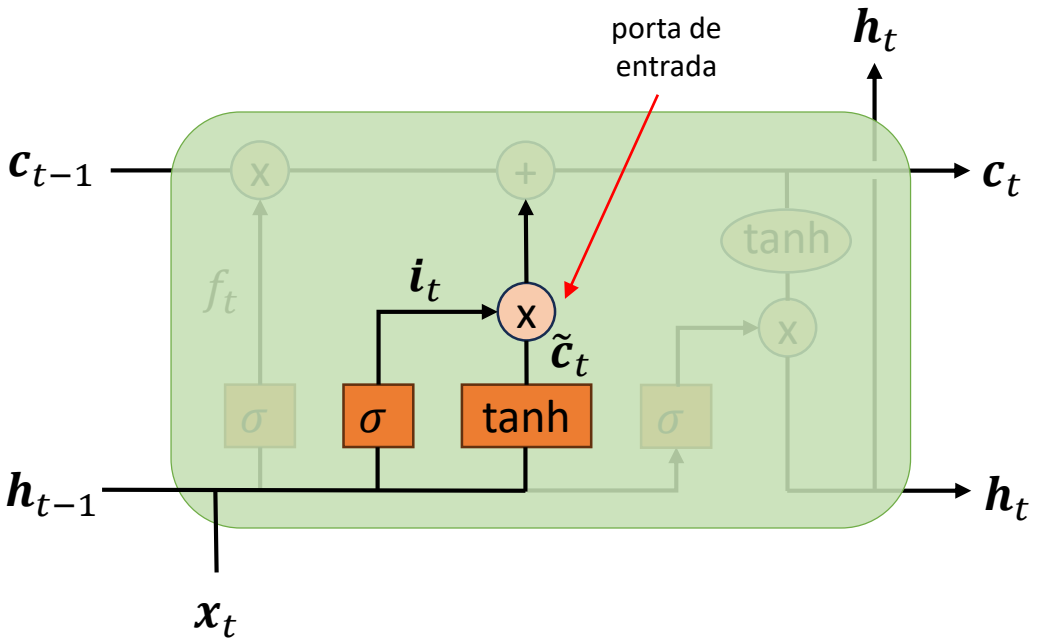

h_t
c_{t-1}
x
+
c_t
tanh
x
x
σ
σ
tanh
σ
h_{t-1}
h_t
x_t

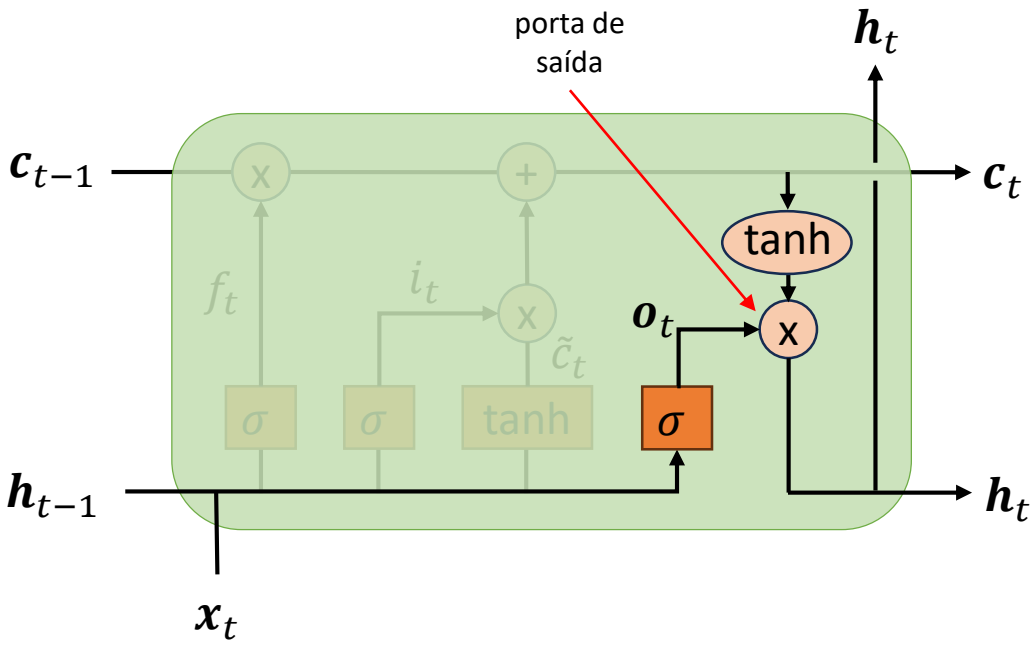

x_t
x
y_t
$g_t = \sigma([h_{t-1}, x_t]W + b)$
σ
porta

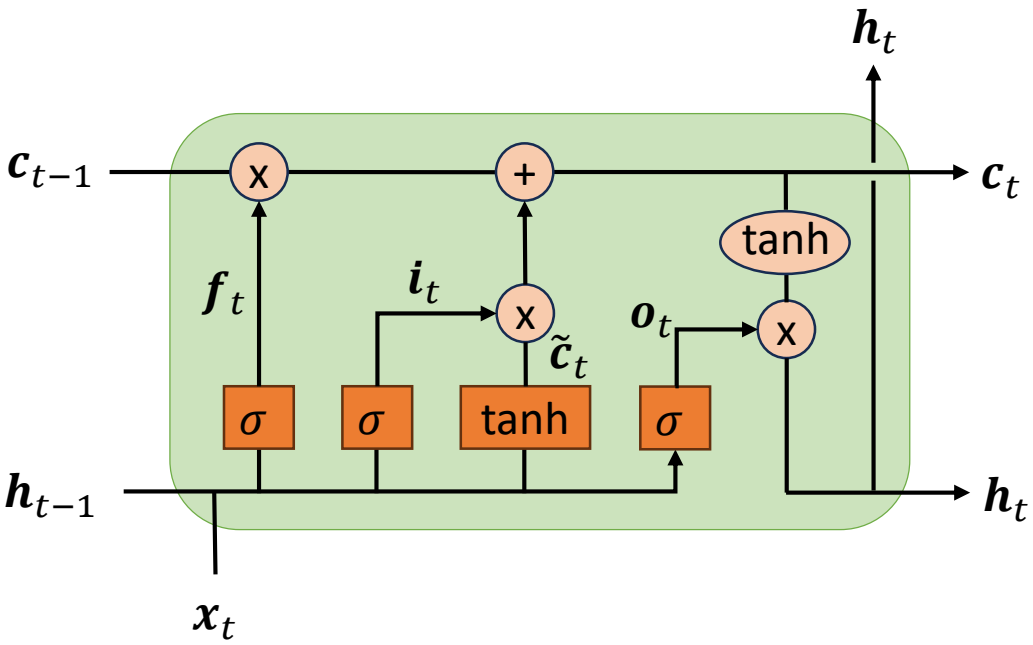

h_t
c_{t-1}
x
+
c_t
tanh
f_t
i_t
o_t
x
x
$\tilde{c}_t$
σ
σ
tanh
σ
h_{t-1}
h_t
x_t

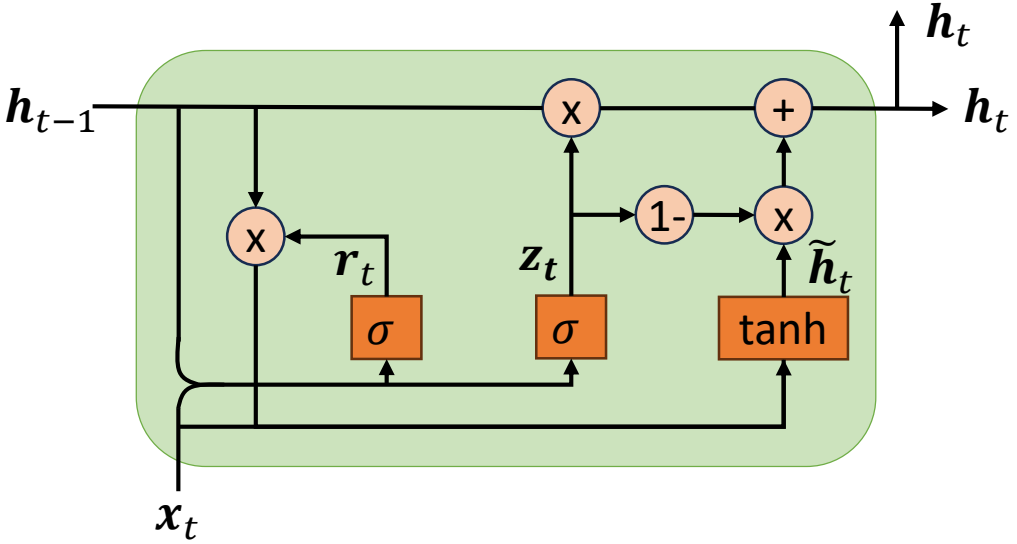

porta de
esquecimento
h_t
c_{t-1}
x
+
c_t
tanh
f_t
x
x
σ
σ
tanh
σ
h_{t-1}
h_t
x_t

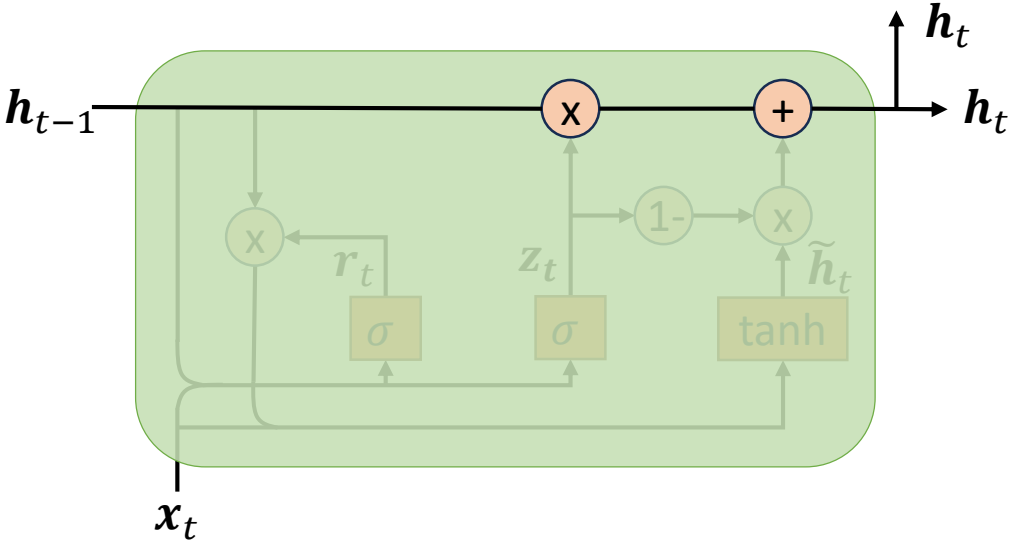

h_t
c_{t-1}
x
+
c_t
tanh
f_t
i_t
x
$\tilde{c}_t$
x
σ
σ
tanh
σ
h_{t-1}
h_t
x_t

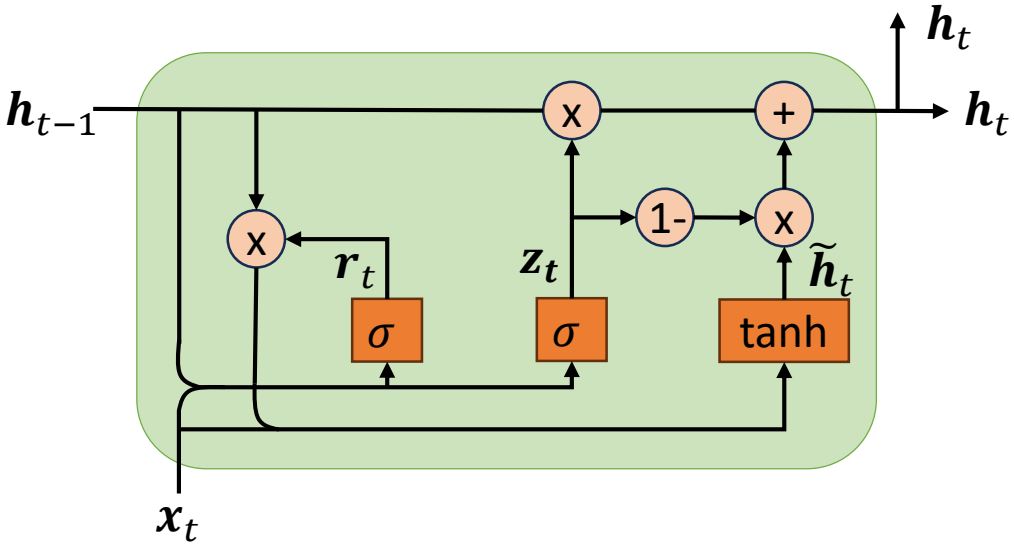

porta de
entrada
h_t
c_{t-1}
c_t
x
+
tanh
f_t
i_t
x
$\tilde{c}_t$
σ
σ
tanh
σ
x
h_{t-1}
h_t
x_t

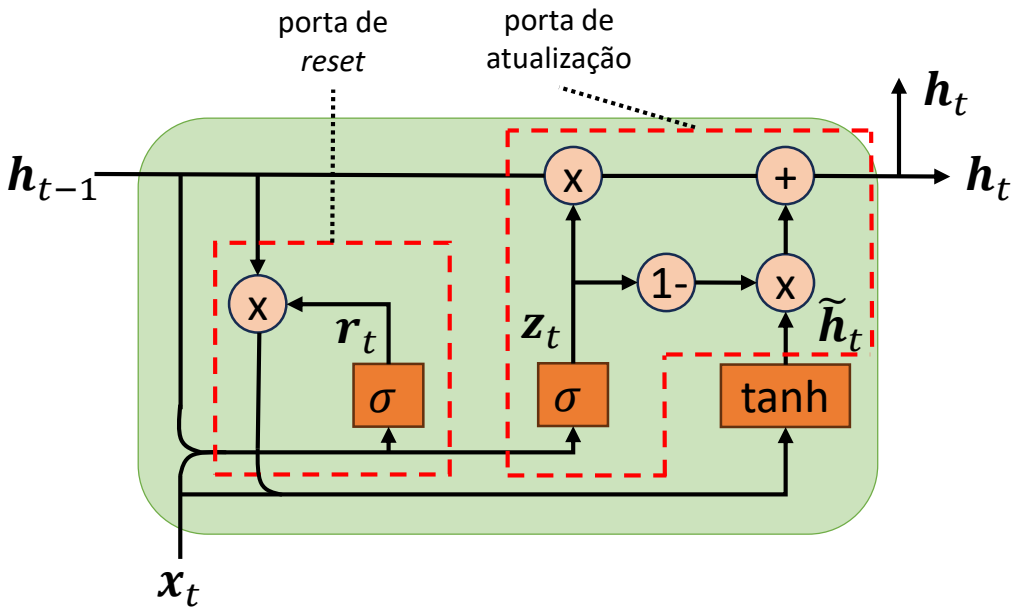

porta de
saída
h_t
c_{t-1}
c_t
x
+
tanh
f_t
i_t
x
o_t
x
$\tilde{c}_t$
σ
σ
tanh
σ
h_{t-1}
h_t
x_t


h_t
c_{t-1}
x
+
c_t
tanh
f_t
i_t
x
o_t
x
$\tilde{c}_t$
σ
σ
tanh
σ
h_{t-1}
h_t
x_t

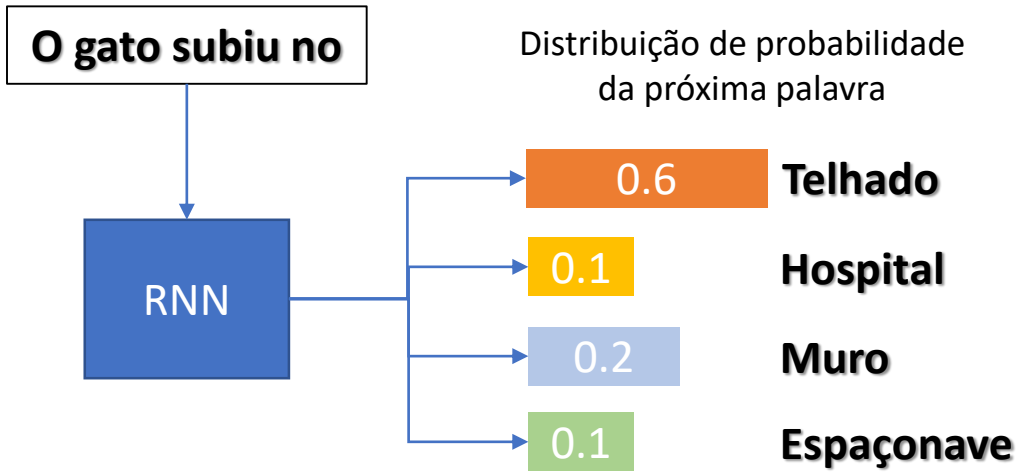

h_t
h_{t-1}
h_t
x
+
x
r_t
1-
z_t
x
$\tilde{h}_t$
σ
σ
tanh
x_t


h_t
h_{t-1}
h_t
x
+
x
r_t
1-
z_t
x
$\tilde{h}_t$
σ
σ
tanh
x_t


h_t
h_{t-1}
h_t
x
+
x
r_t
1-
z_t
x
$\tilde{h}_t$
σ
σ
tanh
x_t


porta de
reset
porta de
atualização
h_t
h_{t-1}
x
+
h_t
x
1-
x
r_t
z_t
$\tilde{h}_t$
σ
σ
tanh
x_t

Palavras anteriores (contexto)



A palavra com maior
probabilidade é escolhida